ZM.

DOCUMENTATION TECHNIQUE
ÉDITION 2026 · V1.0

GUIDE DE MAINTENANCE & DE DÉPLOIEMENT

Le portfolio, expliqué *de A à Z.*

Architecture, rôle de chaque fichier, mise à jour des contenus, ajout de projets, administration, assistant IA et déploiement complet sur **Vercel + Turso**.

Projet
Portfolio — Zo Mahefa RANAIVO

Site public
zo-mahefa-it.vercel.app

Stack
Next.js 16 · React 19 · Tailwind v4 · libSQL

Usage
Maintenance, évolution, déploiement

Document interne. Aucun mot de passe, jeton Turso ou clé API réel n'est reproduit dans ce PDF : les secrets restent dans `.env.local` et dans les variables d'environnement Vercel.

MODE D’EMPLOI

Ce guide est fait pour être ouvert au moment où vous en avez besoin.

Vous n’avez pas besoin de lire 40 pages avant de modifier une phrase. Le tableau ci-dessous vous envoie directement au bon fichier et à la bonne recette. Les chapitres 1 à 4 expliquent le fonctionnement général ; les chapitres 5 à 8 sont des recettes de modification ; le chapitre 11 est la procédure de déploiement pas à pas.

Ce que vous voulez faire	Fichier à ouvrir	Chapitre
Changer nom, e-mail, téléphone, LinkedIn, GitHub, CV	`src/lib/content.ts` → `site`	5.1
Changer les textes du hero, les statistiques	`content.ts` → `hero` (FR et EN)	5.2
Changer les titres de sections	`content.ts` → `sections`	5.3
Modifier un projet vedette	`content.ts` → `featuredProjects`	5.4 / 6
Ajouter un projet vedette	`content.ts` + `public/projects/`	6.1
Ajouter un projet d’archive	`content.ts` → `archive.projects`	6.2
Modifier stack, QA, méthode, pipeline	`content.ts` → `stack`, `qa`, `process`, `pipeline`	5.6 / 5.7
Modifier parcours, formation, certifications	`content.ts` → `about`	5.8
Modifier les témoignages	`content.ts` → `testimonials`	5.9
Modifier le formulaire de contact	`content.ts` → `contact`	5.10
Modifier le bandeau de technologies qui défile	`src/components/site/stack-marquee.tsx`	5.6
Modifier les métadonnées SEO / Open Graph	`src/app/layout.tsx`, `public/og-image.png`	5.14
Changer le CV ou la photo	`public/` + `site.cv`	5.13
Changer le texte de l’assistant IA	`content.ts` → `ai`, `src/lib/assistant-prompt.ts`	8.4
Déployer sur Vercel + Turso	Procédure complète	11
Publier sur LinkedIn	Texte prêt à copier	12

> **RÈGLE D’OR**
> Tout le contenu éditorial vit dans `src/lib/content.ts`, en deux exemplaires : `fr` et `en`. Si vous modifiez une phrase d’un côté, modifiez la même clé de l’autre côté. Le test `src/lib/content.test.ts` vérifie que les deux langues ont exactement la même structure et aucun texte vide.

Sommaire

01	Comprendre le portfolio	Architecture & principes
02	Arborescence commentée	Où vit chaque chose
03	Chaque fichier, son rôle	Référence complète
04	Flux de fonctionnement	Rendu, i18n, formulaires, IA
05	Modifier les contenus	Recettes concrètes
06	Ajouter un projet	Vedette & archive
07	Admin, base de données, sécurité	Messages & audience
08	Assistant IA	Groq / Gemini
09	Qualité, tests & sécurité	Vérifier avant de livrer
10	Développement local	Installation & commandes
11	Déploiement Vercel + Turso	Guide pas à pas
12	Publication LinkedIn	Texte prêt à copier
13	Annexes	Checklists & commandes

Conventions : les chemins sont relatifs à la racine du projet. Les blocs sur fond sombre sont des commandes ou des extraits de code. Les encadrés orange signalent un piège ; les encadrés bleus, une astuce.

CHAPITRE 01

Comprendre le portfolio

Le portfolio est une application **Next.js** qui sert une page unique, bilingue français/anglais, enrichie de quelques fonctions serveur : formulaire de contact, suivi d'audience, administration privée et assistant IA. Le contenu éditorial est **statique** ; seuls les messages et les visites passent par une base de données.

1.1 Ce que fait réellement le site

La page d'accueil raconte un parcours professionnel en six sections : projets, expertise, méthode, parcours, témoignages et contact. Autour de cette page, quatre capacités techniques tournent :

- **Un sélecteur de langue FR/EN** qui change instantanément tous les textes sans recharger la page.
- **Un formulaire de contact** qui écrit dans la table `messages`, avec repli vers l'e-mail si la base est indisponible.
- **Un compteur de visites** qui écrit dans la table `page_views`, consultable dans `/admin`.
- **Un assistant IA** qui répond aux questions sur le parcours à partir d'un prompt construit avec le contenu public du portfolio.

> **À RETENIR**
> Les projets, expériences, certifications et témoignages ne sont **pas** stockés en base. Ils sont écrits dans `src/lib/content.ts`, compilés avec le site et affichés immédiatement. La base ne contient que deux choses : les messages reçus et les visites.

1.2 La stack technique

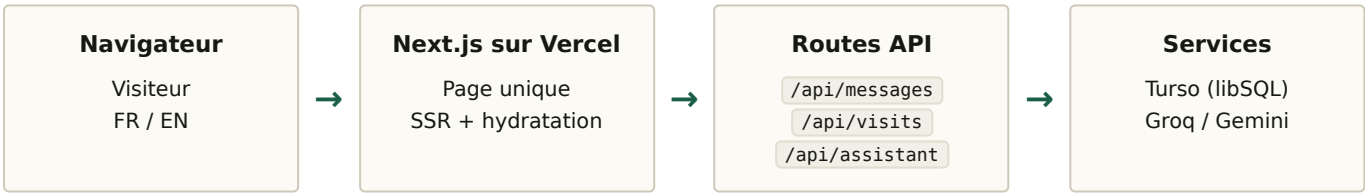
Technologie	Version	Rôle dans le projet
Next.js (App Router)	16.4.x	Framework principal : pages, routes API, rendu serveur, métadonnées SEO, redirections.
React	19.2.x	Interface et composants interactifs (hero, galerie, formulaire, admin, assistant).
TypeScript	5.x	Typage strict du contenu, des réponses API et des composants.
Tailwind CSS	v4	Utilitaires de style, surtout dans l'administration et les composants `ui/`.
CSS global	—	`src/app/globals.css` contient le design system, les sections, les animations et le responsive.
Framer Motion	12.x	Animations d'apparition, respect de `prefers-reduced-motion`.
next-themes	0.4.x	Thème clair/sombre, classe `dark` sur `<html>`.
libSQL / Turso	0.17.x	Base de données des messages et des visites ; SQLite local en développement.
Groq / Google Gemini	API	Fournisseur du modèle derrière l'assistant IA.

Technologie	Version	Rôle dans le projet
Vitest	4.x	Tests unitaires du contenu, de l’authentification et des règles de validation.
Vercel	—	Hébergement, build, variables d’environnement, domaine.

1.3 Statique ou dynamique ?

Élément	Nature	Où il vit
Textes, projets, expériences, témoignages	Statique	`src/lib/content.ts`, compilé dans le bundle.
Images, CV, attestations	Statique	`public/`. Servis par Next.js/Vercel.
Langue, thème	Client	`localStorage` : clés `zm_locale` et `theme`.
Messages du formulaire	Dynamique	Table `messages` (Turso en production, SQLite local sinon).
Visites	Dynamique	Table `page_views`.
Réponses de l’assistant	Dynamique	Route `/api/assistant` → Groq ou Gemini. Aucun stockage.
Session admin	Cookie	Cookie `zm_admin_session`, signé HMAC, 8 heures.

1.4 Vue d’ensemble de l’architecture



Le rendu initial de la page est produit par le serveur Next.js. Les composants interactifs (header, hero, galeries, formulaire, assistant) sont ensuite hydratés côté client. La page reste donc lisible et indexable même si JavaScript est lent à charger ; les animations et les interactions arrivent après.

Les quatre chemins réseau à connaître

Route	Méthode	Accès	Ce qu’elle fait
`/api/messages`	POST	Public (rate limit)	Enregistre un message du formulaire. Validation, honeypot côté client, repli e-mail.
`/api/messages`	GET	Admin	Liste tous les messages, du plus récent au plus ancien.
`/api/messages/[id]`	DELETE	Admin	Supprime un message par son identifiant.
`/api/visits`	POST	Public (rate limit)	Enregistre une vue de page. Ignore les robots et `/admin`.
`/api/visits`	GET	Admin	Renvoie les statistiques agrégées d’audience.
`/api/assistant`	POST	Public (rate limit)	Transmet les messages au fournisseur IA et renvoie la réponse.
`/api/admin/login`	POST	Public	Vérifie le mot de passe et pose le cookie de session.
`/api/admin/logout`	POST	Public	Efface le cookie de session.

Route	Méthode	Accès	Ce qu’elle fait
`/api/admin/session`	GET	Cookie	Indique si la session courante est valide.

1.5 Les trois vérités à retenir

01

Le contenu est centralisé

Une seule source : `content.ts`. Pas de base de données pour les projets, pas de CMS. C’est volontaire : simplicité, rapidité, versionnement Git.

02

La base est minimale

Deux tables : `messages` et `page_views`. Si Turso tombe, le site continue d’afficher le contenu ; le formulaire propose l’e-mail.

03

Les secrets restent dehors

Mot de passe admin, jeton Turso et clé IA vivent dans `.env.local` et dans Vercel. Jamais dans Git, jamais dans ce PDF.

PIÈGE CLASSIQUE

Ne cherchez pas les projets dans la base de données ni dans un fichier JSON : ils sont dans `src/lib/content.ts`. Et ne modifiez pas `src/types/index.ts` en croyant changer les projets : ce fichier ne sert qu’au type `Message` de l’administration ; les types `Project`, `Skill` et `Experience` qu’il contient sont d’anciens types non utilisés par la page d’accueil.

CHAPITRE 02

Arborescence commentée

Voici l’arborescence réelle du dépôt, sans `node_modules`, sans `.next` et sans les fichiers générés. Chaque ligne importante est commentée pour que vous sachiez où chercher avant d’ouvrir un fichier.

```
portfolio/
├── src/
│   ├── app/                        # Routes, pages et API (App Router)
│   │   ├── layout.tsx                  # SEO global, providers, header/footer, IA, tracker
│   │   ├── page.tsx                    # Page d'accueil : ordre des sections
│   │   ├── globals.css                 # Design system + styles de toutes les sections
│   │   ├── icon.svg                    # Favicon servi sur /icon.svg
│   │   ├── manifest.ts                 # Manifest PWA
│   │   ├── robots.ts                   # /robots.txt
│   │   ├── sitemap.ts                  # /sitemap.xml
│   │   ├── admin/
│   │   │   ├── layout.tsx              # Métadonnées noindex de l'admin
│   │   │   └── page.tsx                # UI admin : connexion, messages, audience
│   │   ├── api/
│   │   │   ├── admin/login/route.ts    # Connexion par mot de passe
│   │   │   ├── admin/logout/route.ts   # Déconnexion
│   │   │   ├── admin/session/route.ts  # Vérification de session
│   │   │   ├── assistant/route.ts      # Proxy Groq / Gemini
│   │   │   ├── messages/route.ts       # GET (admin) + POST (public)
│   │   │   ├── messages/[id]/route.ts  # DELETE (admin)
│   │   │   └── visits/route.ts         # POST public + GET stats admin
│   │   ├── competences/page.tsx        # Redirection 307 vers /#expertise
│   │   ├── contact/page.tsx            # Redirection 307 vers /#contact
│   │   └── projets/page.tsx            # Redirection 307 vers /#projets
│   ├── components/
│   │   ├── icons.tsx                   # Icônes GitHub / LinkedIn
│   │   ├── language-provider.tsx       # Contexte FR/EN + persistance
│   │   ├── theme-provider.tsx          # Thème clair/sombre (next-themes)
│   │   ├── site/                       # Composants de la page publique
│   │   │   ├── header.tsx              # Navigation, progression, menu mobile
│   │   │   ├── hero.tsx                # Première section + compteurs
│   │   │   ├── stack-marquee.tsx       # Bandeau technologies (liste locale)
│   │   │   ├── selected-work.tsx       # Projets vedettes + archive
│   │   │   ├── project-gallery.tsx     # Carrousel de captures
│   │   │   ├── project-visual.tsx      # Schémas SVG si pas de captures
│   │   │   ├── capabilities.tsx        # Expertise, stack, QA
│   │   │   ├── process.tsx             # Méthode en 4 étapes
│   │   │   ├── pipeline.tsx            # Chaîne de livraison
│   │   │   ├── about.tsx               # Parcours, formation, certifications
│   │   │   ├── testimonials.tsx        # Témoignages clients
│   │   │   ├── contact-section.tsx     # Coordonnées, parcours recruteur/client
│   │   │   ├── contact-form.tsx        # Formulaire + validation + repli mailto
│   │   │   ├── footer.tsx              # Pied de page
│   │   │   ├── ai-assistant.tsx        # Widget flottant de l'assistant
│   │   │   ├── visit-tracker.tsx       # Envoi d'une vue à /api/visits
│   │   │   ├── copy-email.tsx          # Bouton copier l'e-mail
│   │   │   ├── language-toggle.tsx     # Bouton FR/EN
│   │   │   ├── theme-toggle.tsx        # Bouton clair/sombre
│   │   │   ├── local-clock.tsx         # Heure locale de Fianarantsoa
│   │   │   ├── count-up.tsx            # Compteur animé
│   │   │   ├── typewriter.tsx          # Rôles animés
│   │   │   ├── reveal.tsx              # Apparition au scroll
│   │   │   └── section-heading.tsx     # Titre de section réutilisable
│   │   └── ui/sonner.tsx               # Toaster (notifications)
│   ├── lib/
│   │   ├── content.ts                  # ★ TOUT le contenu FR/EN + types
│   │   ├── assistant-prompt.ts         # Prompt système de l'assistant
│   │   ├── db.ts                       # Schéma + requêtes libSQL
│   │   ├── messages.ts                 # Mapping messages ↔ base
│   │   ├── auth.ts                     # Mot de passe, HMAC, cookie, rate limit
│   │   ├── content.test.ts             # Parité FR/EN + textes non vides
│   │   ├── auth.test.ts                # Mot de passe et robustesse
│   │   └── validation.test.ts          # Regex e-mail/téléphone (inline)
│   ├── types//index.ts          # Type Message + anciens types non utilisés
│   └── test//setup.ts           # Configuration Vitest / jest-dom
├── public/
```

```
│   ├── cv-zomahefa-ranaivo-2026-10.pdf # CV téléchargé par le bouton CV
│   ├── photo-cv.png                    # Portrait du hero
│   ├── og-image.png                    # Image de partage 1200×630
│   ├── apple-icon.png                  # Icône Apple
│   ├── certificatSpringBoot.pdf        # Attestation Spring Boot
│   ├── certifcatSping-boot2.jpeg       # Vignette de l'attestation
│   ├── uml-certification-zo.pdf        # Attestation UML
│   ├── uml-certification.jpeg          # Vignette de l'attestation
│   ├── google81dfecfc5d73987c.html     # Vérification Google Search Console
│   ├── profile-en.pdf                  # Ancien CV EN, non référencé
│   ├── fonts/                          # Polices locales (Geist, Instrument Serif)
│   └── projects/                       # Captures des projets
├── scripts//init-db.mjs          # npm run db:init (Turso ou SQLite)
├── docs/                          # Ce guide (HTML source + PDF)
├── .env.example                        # Modèle de variables, versionné
├── .env.local                          # Secrets locaux, NON versionné
├── next.config.ts                      # En-têtes de sécurité, images
├── package.json                        # Scripts et dépendances
├── tsconfig.json                       # Alias @/ → src/
├── vitest.config.ts                    # Configuration des tests
├── Dockerfile / run.sh                 # Déploiement Docker optionnel
├── README.md                           # Résumé du projet
└── .gitignore                          # Exclut .env*, /data, .next, etc.
```

2.1 Les dossiers en une phrase

Dossier	Responsabilité
`src/app/`	Le routage Next.js : une page d'accueil, trois redirections, l'admin et les routes API.
`src/components/site/`	Les composants visuels de la page publique. Chacun reçoit ses textes du contexte de langue.
`src/components/`	Les providers transverses (langue, thème) et les icônes.
`src/lib/`	La logique : contenu, base de données, authentification, prompt IA. C'est ici que vivent les règles métier.
`src/types/`	Le type `Message` utilisé par l'admin. Les autres types sont historiques.
`public/`	Les fichiers servis tels quels : CV, images, polices, attestations.
`scripts/`	Les scripts d'exploitation, ici l'initialisation de la base.
`docs/`	La documentation du projet et ce PDF.

ASTUCE DE NAVIGATION

Dans votre éditeur, utilisez la recherche globale sur les clés du contenu : `featuredProjects`, `testimonials`, `certifications`, `stats:`. Vous arriverez directement à la bonne zone de `content.ts`, sans dépendre des numéros de ligne qui changent à chaque modification.

CHAPITRE 03

Chaque fichier, son rôle

Ce chapitre est la référence à garder ouverte. Pour chaque fichier, il indique **ce qu'il fait** et **quand le modifier**. Les fichiers critiques — `content.ts`, `layout.tsx`, `db.ts`, `auth.ts` — ont une fiche plus détaillée.

3.1 Racine du projet et configuration

Fichier	Rôle	Quand le modifier
`package.json`	Nom, scripts (`dev`, `build`, `start`, `lint`, `test`, `db:init`), dépendances et versions.	Ajouter une librairie, un script, changer une version.
`package-lock.json`	Verrou exact des dépendances installées.	Jamais à la main : généré par `npm install`.
`next.config.ts`	En-têtes de sécurité (CSP, HSTS, X-Frame-Options…), optimisation des images, compression.	Ajouter un domaine d'image ou ajuster une politique de sécurité.
`tsconfig.json`	TypeScript strict, alias `@/` vers `src/`.	Rarement. Pour ajouter un alias ou changer la cible.
`postcss.config.mjs`	Branche Tailwind v4 sur PostCSS.	Quasi jamais.
`eslint.config.mjs`	Règles ESLint Next.js (core-web-vitals + TypeScript).	Ajouter une règle ou ignorer un dossier.
`vitest.config.ts`	Environnement de test `jsdom`, alias `@`, setup.	Ajouter un setup ou changer l'environnement.
`.env.example`	Modèle documenté des variables d'environnement, versionné.	À chaque nouvelle variable. Ne jamais y mettre de valeur réelle.
`.env.local`	Vos secrets locaux (admin, Turso, IA).	En local uniquement. Ignoré par Git.
`.gitignore`	Exclut `.env*` (sauf `.env.example`), `/data`, `.next`, `node_modules`…	Ajouter un dossier généré ou secret.
`README.md`	Résumé du projet, stack, commandes, variables, admin, IA, sécurité.	Changer une procédure ou une variable.
`Dockerfile`	Image Node 20 Alpine, build Next.js, démarrage `npm run start`.	Déploiement Docker auto-hébergé (optionnel).
`run.sh`	Script de lancement Docker local avec `.env.local` et volume `data/`.	Déploiement Docker local (optionnel).
`.dockerignore`	Exclut les fichiers inutiles de l'image Docker.	Ajouter un gros dossier à exclure.
`components.json`	Configuration shadcn/ui (alias, style).	Ajout de composants shadcn.

3.2 Le cœur : `src/lib/content.ts`

C'est le fichier le plus important du projet. Il contient :

- `siteUrl` : l'URL publique, lue depuis `NEXT_PUBLIC_SITE_URL` avec un domaine par défaut.

- `site` : les informations neutres, identiques en français et en anglais (nom, e-mail, téléphone, réseaux, CV, coordonnées, fuseau).
- Les **types** de toutes les structures : `FeaturedProject`, `ArchiveProject`, `Practice`, `StackLevel`, `QaCategory`, `Experience`, `Certification`, `Testimonial`, `SiteContent`…
- `const fr: SiteContent` : tout le contenu français.
- `const en: SiteContent` : tout le contenu anglais, avec la même structure.
- `contentByLocale`, `defaultContent`, `getContent(locale)` et `mailtoForProject(title)`.

Ordre réel du fichier, de haut en bas :

```
siteUrl
site
types (lignes ~24 à ~403)
const fr: SiteContent = { ... }      # contenu français
const en: SiteContent = { ... }      # contenu anglais
export const contentByLocale = { fr, en }
export const defaultContent = fr
export function getContent(locale)
export function mailtoForProject(title)
```

> **NE PAS SE FIER AUX NUMÉROS DE LIGNE**
> Le fichier fait plus de 2 000 lignes et bouge à chaque ajout. Utilisez les ancres textuelles : `const fr: SiteContent`, `const en: SiteContent`, `featuredProjects:`, `archive:`, `about:`, `testimonials:`.

Les blocs de `SiteContent`

Bloc	Sections du site concernées
`availability`, `workModes`, `location`, `roles`	Disponibilité affichée dans le header, le hero, le contact et le prompt IA.
`nav`	Menu du header et du footer. Les `id` doivent correspondre aux `id` des sections.
`hero`	Section d'ouverture : titres, intro, boutons, statistiques, portrait, notes.
`sections`	Titres et intros des six sections (projets, expertise, méthode, parcours, témoignages, contact).
`featuredProjects`	Les projets détaillés (Signaleo, E-Fandrosoana…).
`archive`	Les autres projets, en lignes compactes.
`practices`	Les trois pôles : web/mobile, DevOps, QA.
`stack`	Les niveaux de maîtrise réels.
`qa`	Le bloc qualité : catégories, outils, mindset, citation.
`process`	La méthode en quatre étapes + engagements.
`pipeline`	La chaîne commit → test → build → ship → observe.
`about`	Parcours, expériences, formation, certifications, savoir-être, langues.
`testimonials`	Les témoignages clients et leur note.
`contact`	Coordonnées, parcours recruteur/client, libellés du formulaire, types de besoin.
`footer`	Accroche, libellés, CV, retour en haut, mentions.
`ui`	Tous les micro-libellés : boutons, galerie, accessibilité, thème, langue, menu.
`ai`	Textes du widget assistant : titre, accueil, suggestions, erreurs, disclaimer.

3.3 Les fichiers de `src/app/`

Fichier	Rôle	Quand le modifier
`layout.tsx`	Enveloppe globale : balises `<html>` et `<body>`, métadonnées SEO, JSON-LD, préchargement des polices, providers thème/langue, header, footer, tracker, assistant, toaster.	Changer le titre SEO, la description, les mots-clés, l'Open Graph, le JSON-LD, les polices préchargées.
`page.tsx`	Compose la page d'accueil dans l'ordre : Hero, StackMarquee, SelectedWork, Capabilities, Process, Pipeline, About, Testimonials, ContactSection.	Réordonner ou retirer une section.
`globals.css`	Design system complet : variables de couleurs, typographies, layout, styles de chaque section, animations, responsive, thème sombre.	Changer l'apparence, les espacements, les couleurs, une animation. Prudence : 5 000+ lignes.
`icon.svg`	Favicon moderne servi sur `/icon.svg`.	Changer le logo de l'onglet.
`manifest.ts`	Manifest PWA : nom, description, couleurs, icônes, langue.	Changer le nom de l'application ou les icônes.
`robots.ts`	Génère `/robots.txt` : autorise le site, exclut `/admin/` et `/api/`, indique le sitemap.	Changer les règles d'indexation.
`sitemap.ts`	Génère `/sitemap.xml` avec l'URL racine et une fréquence mensuelle.	Ajouter une vraie page indexable (rare).
`admin/layout.tsx`	Métadonnées `noindex, nofollow, nocache` pour l'administration.	Quasi jamais.
`admin/page.tsx`	Interface client complète : écran de connexion, chargement des messages et des stats, onglets Messages/Audience, recherche, suppression, déconnexion.	Changer l'interface d'administration.
`api/admin/login/route.ts`	Vérifie le mot de passe, limite à 6 tentatives par 15 minutes et par IP, pose le cookie HMAC 8 h.	Changer la durée de session ou la politique de limitation.
`api/admin/logout/route.ts`	Efface le cookie de session.	Quasi jamais.
`api/admin/session/route.ts`	Répond `authenticated: true` si le cookie est valide, 401 sinon.	Quasi jamais.
`api/messages/route.ts`	POST public : validation (nom, e-mail, téléphone, message ≥ 10 caractères), honeypot côté client, rate limit 5/10 min, insertion. GET admin : liste.	Changer les règles de validation ou les limites.
`api/messages/[id]/route.ts`	DELETE admin : supprime un message par identifiant.	Quasi jamais.
`api/visits/route.ts`	POST public : ignore robots et `/admin`, déduit appareil et pays, enregistre. GET admin : statistiques agrégées.	Changer les métriques suivies ou ignorer d'autres chemins.
`api/assistant/route.ts`	Proxy serveur vers Groq ou Gemini : validation des messages, 8 derniers messages, 1 200 caractères max, rate limit 25/10 min, repli propre.	Changer de modèle, de fournisseur, de limite ou de paramètres.
`competences/page.tsx`, `contact/page.tsx`, `projets/page.tsx`	Anciennes URLs conservées : redirection 307 vers l'ancre correspondante, `noindex`, canonical vers l'ancre.	Ne pas y remettre de contenu : elles existent pour éviter les liens morts.

3.4 Les routes API en détail

`/api/messages` — recevoir un message

1. Lecture de l'IP via `x-forwarded-for` (premier segment) ou `unknown`.
2. Limitation : 5 requêtes par 10 minutes et par IP (en mémoire de l'instance).
3. Nettoyage : `trim` puis coupe à 120 (nom), 180 (e-mail), 30 (téléphone), 5 000 (message), 80 (type de projet).
4. Validation : nom requis, e-mail regex, téléphone regex si fourni, message ≥ 10 caractères.
5. Si un `projectType` est fourni, il est préfixé au message : `Type de besoin : …`.
6. Insertion en base, réponse 201 avec le message créé.
7. En cas d'erreur base : réponse 503 avec `fallback: "email"`, et le formulaire propose un lien `mailto:` prérempli.

`/api/visits` — mesurer l'audience

1. Limitation : 60 requêtes par 10 minutes et par IP ; au-delà, réponse 204 silencieuse.
2. Filtrage des robots : `bot`, `crawler`, `spider`, `headless`, `lighthouse`.
3. Chemin nettoyé (200 caractères), doit commencer par `/` et ne pas commencer par `/admin`.
4. Référent (300 caractères), identifiant visiteur (80 caractères, `anonymous` par défaut).
5. Pays lu dans `x-vercel-ip-country` ou `cf-ipcountry`.
6. Appareil déduit du `user-agent` : Tablet, Mobile, Desktop.
7. Insertion puis réponse 204, sans corps.

`/api/assistant` — répondre aux questions

1. Rate limit : 25 requêtes par 10 minutes et par IP.
2. Résolution du fournisseur : `AI_PROVIDER` explicite si la clé existe, sinon Groq, sinon Google, sinon 503.
3. Nettoyage : garde les 8 derniers messages, tronque chaque contenu à 1 200 caractères, ignore les rôles invalides.
4. Construction du prompt système via `buildAssistantSystemPrompt()`.
5. Appel du fournisseur, réponse JSON `{ reply, provider }`, en-tête `no-store`.

3.5 Les composants de la page publique

Composant	Responsabilité	Détail utile
`header.tsx`	Barre de navigation fixe, logo, liens d'ancrage, barre de progression de lecture, section active, menu mobile, thème, langue, heure locale.	Les liens viennent de `c.nav` ; les `id` doivent exister dans les sections.
`hero.tsx`	Ouverture : nom, rôles animés, titre, intro, boutons, statistiques, portrait, notes.	Les statistiques utilisent `CountUp` avec `value`, `pad`, `prefix`, `suffix`.
`stack-marquee.tsx`	Bandeau défilant des technologies.	**Liste locale codée en dur**, séparée de `content.ts`.
`selected-work.tsx`	Projets vedettes (étude de cas, stack, liens) puis archive.	Si `gallery` existe → carrousel ; sinon → `ProjectVisual`.

Composant	Responsabilité	Détail utile
`project-gallery.tsx`	Carrousel de captures avec autoplay 5 s, pause au survol, flèches, clavier, points, compteur.	Désactive l'autoplay si `prefers-reduced-motion`.
`project-visual.tsx`	Schéma SVG stylisé pour les projets sans captures : `incident`, `erp`, `pipeline`.	Un nouveau type visuel doit être ajouté ici.
`capabilities.tsx`	Trois pôles d'expertise, stack par niveaux, bloc QA complet.	Lit `practices`, `stack`, `qa`.
`process.tsx`	Méthode en quatre étapes et engagements.	Lit `process.steps` et `process.engagements`.
`pipeline.tsx`	Chaîne de livraison illustrée, section sombre.	Lit `pipeline.steps`.
`about.tsx`	Parcours, expériences, formation, certifications, savoir-être.	Les certifications ouvrent un PDF dans un nouvel onglet.
`testimonials.tsx`	Cartes de témoignages avec nom, entreprise, contexte, téléphone.	La clé React est `testimonial.name` : éviter les doublons.
`contact-section.tsx`	Coordonnées, copie d'e-mail, parcours recruteur/client, réseaux, CV, formulaire.	Deux liens `mailto:` avec sujets préremplis.
`contact-form.tsx`	Formulaire contrôlé, validation, honeypot, consentement, envoi API, succès, erreur, repli mailto.	Le champ `website` est un honeypot invisible.
`footer.tsx`	Identité, coordonnées GPS, heure locale, navigation, contact, réseaux, retour en haut.	Lit `footer` et `nav`.
`ai-assistant.tsx`	Bouton flottant, panneau de discussion, suggestions, historique local, envoi, erreur, reset.	Masqué sur `/admin`.
`visit-tracker.tsx`	Envoie une vue à chaque changement de chemin, sauf `/admin`.	Identifiant visiteur dans `localStorage` (`zm_visitor_id`).
`copy-email.tsx`	Copie l'e-mail dans le presse-papiers avec retour visuel.	Utilise `navigator.clipboard`.
`language-toggle.tsx`	Bascule FR ↔ EN.	Met à jour `document.documentElement.lang`.
`theme-toggle.tsx`	Bascule clair ↔ sombre.	Persistance via next-themes.
`local-clock.tsx`	Affiche l'heure de Fianarantsoa (UTC+3).	Mise à jour toutes les 30 s.
`count-up.tsx`	Compteur animé au premier passage dans le viewport.	Respecte `prefers-reduced-motion`.
`typewriter.tsx`	Rôles qui s'écrivent et s'effacent, et titre de pipeline.	Respecte `prefers-reduced-motion`.
`reveal.tsx`	Animation d'apparition au scroll via Framer Motion.	Utilisé dans toutes les sections.
`section-heading.tsx`	Titre de section réutilisable : index, label, titre, intro, filet.	Garantit la cohérence visuelle.

3.6 Les fichiers de `src/lib/`

Fichier	Rôle	Quand le modifier
`content.ts`	Source unique du contenu FR/EN, types et helpers.	À chaque changement éditorial.

Fichier	Rôle	Quand le modifier
`assistant-prompt.ts`	Construit le prompt système de l'assistant à partir du contenu public : profil, projets, expertise, QA, expériences, formation, certifications, témoignages, stack.	Changer le ton, les règles ou les informations données au modèle.
`db.ts`	Client libSQL, schéma `messages` et `page_views`, création paresseuse des tables, requêtes CRUD et statistiques.	Ajouter une table, une métrique ou changer une requête.
`messages.ts`	Couche métier entre les routes et la base : mapping `created_at` → `createdAt`, ajout, suppression.	Changer la forme d'un message.
`auth.ts`	Vérification du mot de passe en temps constant, signature HMAC du cookie, lecture du cookie, rate limit, détection de mot de passe faible.	Changer la durée de session, la politique de mot de passe ou le nom du cookie.
`content.test.ts`	Vérifie que FR et EN ont la même structure et aucun texte vide.	Ne pas modifier pour faire passer un test : corriger le contenu.
`auth.test.ts`	Vérifie le mot de passe correct/incorrect et la détection de faiblesse.	Ajouter des cas de test.
`validation.test.ts`	Teste des regex e-mail/téléphone définies dans le test lui-même. Il n'existe pas de `validation.ts`.	À savoir : la validation réelle est dans les routes API et le formulaire.

3.7 Les autres fichiers

Fichier	Rôle
`src/types/index.ts`	`Message` est utilisé par l'admin. `Project`, `Skill` et `Experience` sont des types historiques non utilisés par la page d'accueil.
`src/components/icons.tsx`	Icônes SVG GitHub et LinkedIn, réutilisées dans le contact et les projets.
`src/components/language-provider.tsx`	Contexte de langue : `locale`, `setLocale`, `toggleLocale`, `c` (contenu courant). Persiste `zm_locale` et met à jour `lang`.
`src/components/theme-provider.tsx`	Enveloppe next-themes : thème clair par défaut, pas de thème système, classe `dark`.
`src/components/ui/sonner.tsx`	Toaster de notifications, monté dans le layout.
`src/test/setup.ts`	Charge `@testing-library/jest-dom` pour les assertions DOM.
`scripts/init-db.mjs`	Crée les tables et index sur Turso (si `TURSO_DB_URL`) ou en SQLite local, puis affiche un compte rendu.
`public/profile-en.pdf`	Ancien CV anglais, plus référencé par le site. Peut être conservé comme archive ou supprimé.

LE SEUL FICHIER À RETENIR

Pour 90 % des mises à jour de contenu, vous n'ouvrirez que `src/lib/content.ts`. Pour les images et le CV, vous déposerez un fichier dans `public/`. Pour le SEO global, vous ouvrirez `src/app/layout.tsx`. Le reste sert surtout au fonctionnement.

CHAPITRE 04

Flux de fonctionnement

Comprendre un flux évite de modifier le mauvais fichier. Voici les sept parcours qui font vivre le portfolio, de l’affichage d’une page à la réponse de l’assistant.

4.1 Affichage d’une page



Le serveur rend le HTML avec le contenu français par défaut. Au chargement, `LanguageProvider` lit `zm_locale` dans `localStorage` : si le visiteur avait choisi l’anglais, React remplace les textes sans rechargement. Le HTML initial reste en français, ce qui est cohérent avec `lang="fr"` et le référencement.

4.2 Le changement de langue

1. Le visiteur clique sur `LanguageToggle`.
2. `toggleLocale()` bascule `fr` ↔ `en` dans le contexte.
3. `useMemo` recalcule `c = getContent(locale)`.
4. Tous les composants abonnés à `useI18n()` se re-rendent avec les nouveaux textes.
5. Un effet met à jour `document.documentElement.lang` et écrit `zm_locale`.

> **CONSÉQUENCE POUR LE SEO**
> Le site n’a pas deux URLs (`/fr` et `/en`) ni de balises `hreflang`. Le contenu anglais est une bascule cliente. Les moteurs indexent donc principalement la version française. C’est un choix assumé : une seule page, un seul canonical, pas de contenu dupliqué. Si vous voulez un vrai SEO bilingue plus tard, il faudra créer des routes `/en` et des métadonnées dédiées — ce n’est pas le fonctionnement actuel.

4.3 Le thème clair/sombre

1. Un script inline dans `layout.tsx` lit `theme` dans `localStorage` avant le rendu et ajoute la classe `light` ou `dark` sur `<html>`.
2. `ThemeProvider` (next-themes) prend le relais : thème clair par défaut, pas de suivi du système.
3. `ThemeToggle` change la classe et persiste le choix.
4. `globals.css` définit les variables de couleurs pour `:root` et `.dark`.

4.4 Le formulaire de contact



- Le champ `website` est un **honeypot** : s'il est rempli, le formulaire simule un succès sans envoyer.
- Le consentement est obligatoire côté client.
- Le type de besoin est choisi dans `contact.types` et préfixé au message.
- En cas d'échec réseau ou base, le formulaire affiche un lien `mailto:` prérempli : le message n'est pas perdu silencieusement.

4.5 Le suivi d'audience

1. `VisitTracker` s'exécute à chaque changement de `pathname`.
2. Il ignore `/admin`.
3. Il fabrique ou récupère un identifiant visiteur dans `localStorage` (`zm_visitor_id`).
4. Il envoie `{ path, referrer, visitorId }` à `/api/visits` avec `keepalive`.
5. La route ignore les robots, déduit l'appareil et le pays, puis insère dans `page_views`.
6. `/admin` appelle `GET /api/visits` pour obtenir les agrégats.

> **VIE PRIVÉE**
> Aucun cookie publicitaire n'est posé. L'identifiant visiteur est un UUID stocké localement, utilisé uniquement pour compter les visiteurs uniques. Les robots sont filtrés et l'admin n'est pas mesuré. Si vous ajoutez un bandeau de consentement, faites-le avant le tracking.

4.6 L'administration



Le cookie est `HttpOnly`, `SameSite=Strict`, et `Secure` en production. Il contient une date d'expiration et une signature HMAC-SHA256 calculée avec `ADMIN_PASSWORD` comme secret. Changer le mot de passe invalide donc toutes les sessions en cours. Le serveur accepte aussi, pour compatibilité, l'ancien en-tête `Authorization: Bearer <mot_de_passe>`.

L'écran admin ne stocke pas de jeton dans `localStorage` : il s'appuie uniquement sur le cookie. Au premier chargement, il appelle `/api/admin/session`. Si la réponse est 401, l'écran de connexion reste affiché. Si une requête ultérieure renvoie 401, la session est considérée expirée.

4.7 L’assistant IA



Le modèle ne reçoit que des informations publiques : nom, rôles, localisation, disponibilité, projets, expertise, QA, expériences, formation, certifications, témoignages et stack. Il ne voit ni les messages, ni les visites, ni les variables d’environnement, ni le mot de passe admin.

4.8 Le SEO et le partage social

1. `siteUrl` est calculé depuis `NEXT_PUBLIC_SITE_URL` (sans slash final) ou vaut le domaine par défaut.
2. `layout.tsx` utilise cette URL pour `metadataBase`, le canonical, l’Open Graph, Twitter et le JSON-LD.
3. `sitemap.ts` et `robots.ts` utilisent la même valeur.
4. `public/og-image.png` (1200×630) est l’image de partage.
5. Les pages `/projets`, `/contact`, `/competences` sont en `noindex` et redirigent vers les ancres.

> **SI VOUS CHANGEZ DE DOMAINE**
> Modifiez `NEXT_PUBLIC_SITE_URL` dans Vercel, puis **redéployez**. Les métadonnées sont générées au build : sans redéploiement, le canonical, le sitemap et le JSON-LD continueront de pointer vers l’ancien domaine.

CHAPITRE 05

Modifier les contenus

Toutes les recettes de ce chapitre suivent le même principe : ouvrir `src/lib/content.ts`, repérer la clé, modifier la version française **et** la version anglaise, puis lancer les tests. Aucune base de données, aucun CMS, aucune interface d'administration pour le contenu éditorial.

> **PROCÉDURE STANDARD**
>
> 1. Ouvrir `src/lib/content.ts`.
> 2. Chercher la clé (ex. `featuredProjects:`).
> 3. Modifier le bloc dans `const fr`, puis le même bloc dans `const en`.
> 4. Lancer `npm test` : il vérifie la parité des structures et l'absence de texte vide.
> 5. Lancer `npm run build` pour vérifier TypeScript et la compilation.
> 6. Commit + push : Vercel redéploie automatiquement.

5.1 Identité, coordonnées et liens

Ces informations sont **neutres** : elles ne sont pas traduites. Elles vivent dans l'objet `site`, tout en haut de `content.ts`.

```
export const site = {
  name: "Zo Mahefa RANAIVO",
  shortName: "Zo Mahefa",
  initials: "ZM",
  email: "zomahefaranaivo@gmail.com",
  phone: "+261 38 54 422 52",
  phoneHref: "tel:+261385442252",
  location: "Fianarantsoa, Madagascar",
  coordinates: "21°27′S 47°05′E",
  timezone: "Indian/Antananarivo",
  github: "https://github.com/Zomahefa",
  linkedin: "https://www.linkedin.com/in/zo-mahefa-ranaivo-338026346/",
  cv: "/cv-zomahefa-ranaivo-2026-10.pdf",
  responseTime: "Réponse sous 48 h ouvrées",
} as const;
```

Champ	Où il apparaît
`name`, `shortName`, `initials`	Header, hero, footer, métadonnées, JSON-LD.
`email`	Contact, footer, prompt IA, liens `mailto:`.
`phone`, `phoneHref`	Contact, footer, témoignages (téléphone séparé), JSON-LD.
`location`, `coordinates`, `timezone`	Hero, contact, footer, horloge locale, JSON-LD.
`github`, `linkedin`	Contact, footer, projets, JSON-LD `sameAs`.
`cv`	Bouton CV du hero, contact et footer.

COHÉRENCE DU TÉLÉPHONE

`phone` est la version affichée ; `phoneHref` est la version `tel:` sans espaces. Si vous changez l'un, changez l'autre. Le JSON-LD de `layout.tsx` contient aussi le numéro en dur (`+261385442252`) : pensez à le mettre à jour.

5.2 Le hero et les statistiques

Le bloc `hero` existe en français et en anglais. Les champs importants :

```
hero: {
  eyebrowLeft: "PORTFOLIO / ÉDITION 2026",
  eyebrowCenter: "FIANARANTSOA, MADAGASCAR · UTC+3",
  eyebrowRight: "DISPONIBLE POUR COLLABORER",
  kicker: "Ingénieur logiciel",
  title: {
    sr: "Zo Mahefa RANAIVO — Développeur web & mobile, ...",
    line1: "Du premier",
    line2Em: "commit",
    line2After: " au",
    line3Em: "dernier",
    line3After: " test.",
  },
  roleLabel: "RÔLE_",
  intro: "Je conçois des applications web et mobiles, ...",
  primaryCta: "Voir mes preuves",
  secondaryCta: "Parler d'un projet",
  cvCta: "CV PDF",
  stats: [
    { value: 9, pad: 2, label: "projets documentés" },
    { value: 3, pad: 2, label: "pôles d'expertise" },
    { value: 8, pad: 2, label: "dépôts publics" },
    { value: 48, prefix: "<", suffix: "h", label: "délai de réponse" },
  ],
}
```

- `title.line2Em` et `line3Em` sont les mots mis en italique. `line2After` et `line3After` sont le texte qui suit.
- `title.sr` est le titre complet pour les lecteurs d'écran et le SEO. Il doit rester descriptif.
- `stats[].value` est le nombre cible. `pad: 2` affiche `09` au lieu de `9`.
- `prefix` et `suffix` sont optionnels : `<` + `48` + `h` donne `<48h`.
- `roles` (au niveau racine de `SiteContent`) alimente l'animation des rôles sous le titre.

RÈGLE DE CRÉDIBILITÉ

Ne mettez dans les statistiques que des chiffres que vous pouvez défendre : nombre de projets réellement documentés, nombre de dépôts publics, nombre de pôles. Évitez les pourcentages inventés.

5.3 Les titres et intros de sections

Le bloc `sections` contient six objets : `projects`, `expertise`, `method`, `about`, `testimonials`, `contact`. Chacun a `index`, `label`, `title` et `intro`.

```
sections: {
  projects: {
    index: "01",
    label: "Travaux sélectionnés",
    title: "Des preuves, pas des promesses.",
    intro: "Trois projets, trois problèmes réels. ...",
  },
  // ...
}
```

Les index `01` à `06` sont affichés dans les titres. Si vous ajoutez une section, mettez à jour les index, le menu `nav` et l'ordre dans `src/app/page.tsx`.

5.4 Modifier un projet vedette

Les projets vedettes sont dans `featuredProjects`. Chaque objet contient les champs suivants :

Champ	Requis	Rôle
`id`	Oui	Identifiant unique, en kebab-case. Sert de clé React.
`index`	Oui	Numéro affiché (01, 02, 03…).
`title`, `subtitle`	Oui	Nom du projet et accroche courte.
`year`, `category`	Oui	Année et catégorie affichées dans les métadonnées.
`summary`	Oui	Résumé en une phrase.
`context`	Oui	Cadre du projet (licence, freelance, stage…).
`problem`, `role`, `outcome`	Oui	Les trois blocs de l'étude de cas.
`stack`	Oui	Tableau de technologies affichées en tags.
`visual`	Oui	`incident`, `erp` ou `pipeline`.
`gallery`	Non	Captures d'écran. Si présent, remplace le schéma SVG par un carrousel.
`repo`, `demo`, `demoLabel`	Non	Liens vers le code et la démo.
`privateCode`	Non	Si `true`, affiche « Étude de cas sur demande » avec un lien mailto.

Exemple réel : Signaleo

```
{
  id: "signaleo",
  index: "01",
  title: "Signaleo",
  subtitle: "Signalement de pannes · terrain & supervision",
  year: "2025",
  category: "Web app / DevOps",
  summary: "Un outil unique pour signaler une panne, la localiser et suivre sa résolution en temps
réel.",
  context: "Projet de licence L3, conduit du cadrage jusqu'à la mise en production et à la
supervision.",
  problem: "Les interventions étaient suivies sans vue d'ensemble : statuts dispersés, faible
traçabilité...",
  role: "Conception du parcours de signalement, tableau de bord d'administration, suivi des
statuts...",
  outcome: "Une application déployée et observée, avec un flux de livraison reproductible...",
  stack: ["React", "Django", "MySQL", "Docker", "GitHub Actions", "Nginx", "Prometheus", "Grafana"],
  visual: "incident",
  gallery: [
    {
      src: "/projects/signaleo-signalement.png",
      alt: "Capture de l'écran de signalement d'une panne sur Signaleo",
      label: "Signalement d'une panne",
    },
  ],
  repo: "https://github.com/Zomahefa/gestion-de-panne-devops",
  demo: "https://signaleo.onrender.com/",
  demoLabel: "Consulter la démo",
}
```

> **LE CHAMP `VISUAL` EST TYPÉ**
> `ProjectVisualKind` n'accepte que `"incident"`, `"erp"` ou `"pipeline"`. Si vous inventez une valeur comme `"mobile"`, TypeScript refusera le build. Pour un nouveau type, il faut modifier le type dans `content.ts` *et* ajouter le rendu correspondant dans `src/components/site/project-visual.tsx`. Le plus simple reste de fournir une `gallery` : le carrousel remplace le schéma.

5.5 Modifier l'archive

L'archive est le bloc `archive`. Chaque entrée est volontairement courte :

```
archive: {
  kicker: "ARCHIVE / 06",
  title: "Autres projets & labs",
  copy: "Le reste du terrain : desktop, réseau, systèmes et projets académiques...",
  footerNote: "ZÉRO LIEN MORT — LES PROJETS PRIVÉS SONT SIGNALÉS",
  allRepos: "Tous mes dépôts publics",
  projects: [
    {
      title: "Gestion de station essence",
      category: "Desktop · JavaFX",
      year: "2024",
      stack: "JavaFX · PostgreSQL",
      href: "https://github.com/Zomahefa/Gestion-de-station-essence",
      label: "Code",
    },
    {
      title: "Administration Linux & Windows Server",
      category: "Systèmes · ENI",
      year: "2023 — 2024",
      stack: "Active Directory · Bash · iptables",
      label: "Lab",
    },
  ],
}
```

- Si `href` est présent, la ligne est un lien et affiche « Code ».
- Si `href` est absent, la ligne est statique et affiche `label` (« Lab », « Labs »…).
- `kicker` contient le nombre affiché ; pensez à l'ajuster si vous ajoutez des lignes.

5.6 Expertise, stack et QA

`practices` — les trois pôles

```
practices: [
  {
    index: "A",
    title: "Web & mobile",
    intro: "Des interfaces claires et des API solides...",
    deliverables: [
      "Applications métier et tableaux de bord",
      "API REST sécurisées et modélisation des données",
      "Interfaces React/Next.js responsive et accessibles",
      "Applications Flutter/Kotlin selon le besoin",
    ],
    code: "UI / API / DATA",
  },
  // B = DevOps, C = QA
]
```

`stack` — les niveaux réels

```
stack: {
  kicker: "CORE STACK / NIVEAUX RÉELS",
  title: "Ce que je peux défendre en entretien.",
  intro: "Pas de pourcentage inventé...",
  footnote: "UNE TECHNOLOGIE N'EST PAS UNE COMPÉTENCE...",
  levels: [
    { level: "Maîtrisé en production", note: "...", items: ["Java", "Spring Boot", "..."] },
    { level: "Opérationnel", note: "...", items: ["Kubernetes", "..."] },
    { level: "Notions", note: "...", items: ["AWS", "..."] },
  ],
}
```

`qa` — le bloc qualité

```
qa: {
  kicker: "QA / MANUAL & AUTOMATION",
  title: "La qualité, du scénario au pipeline.",
  intro: "...",
  categories: [
    { title: "Gestion & recette", note: "...", tools: ["Jira", "Xray", "Zephyr", "TestRail"] },
    { title: "Tests d'API", note: "...", tools: ["Postman", "Bruno", "Insomnia", "cURL"] },
    // ...
  ],
  mindset: ["QA manuel & SDET", "Esprit critique", "..."],
  footnote: "UN BUG TROUVÉ AVANT LA PRODUCTION COÛTE MOINS CHER...",
}
```

> **DEUX LISTES DE TECHNOLOGIES DIFFÉRENTES**
> Le bandeau défilant du haut de page est codé en dur dans `src/components/site/stack-marquee.tsx` (constante `stack`), et il n'est pas traduit. La liste détaillée par niveaux est dans `content.ts` → `stack.levels`. Si vous ajoutez une technologie importante, mettez-la dans les deux endroits pour rester cohérent.

5.7 Méthode et chaîne de livraison

`process`

```
process: {
  steps: [
    { number: "01", title: "Cadrer", text: "...", deliverable: "Mini-cahier des charges" },
    { number: "02", title: "Construire", text: "...", deliverable: "Incréments testés" },
    { number: "03", title: "Livrer", text: "...", deliverable: "Déploiement reproductible" },
    { number: "04", title: "Veiller", text: "...", deliverable: "Runbook & documentation" },
  ],
  engagements: ["Premier échange de 30 min", "Devis sous 5 jours ouvrés", "Code et documentation
remis", "Réponse sous 48 h ouvrées"],
  engagementsKicker: "ENGAGEMENTS / SIMPLES",
  engagementsTitle: "Des règles lisibles avant même de commencer.",
  cta: "Cadrer un besoin",
}
```

`pipeline`

```
pipeline: {
  label: "Chaîne de livraison",
  title: "Du commit au monitoring.",
  intro: "...",
  principleLabel: "Principe :",
  principle: "un déploiement doit pouvoir être rejoué, observé et annulé...",
  deliverablesLabel: "LIVRABLES / RUNBOOK",
  deliverables: "Pipeline, procédure de rollback et documentation de supervision...",
  cta: "Cadrer un projet",
  footerLeft: "FLUX REJOUABLE · TESTS · LOGS · MÉTRIQUES",
  footerRight: "DOCKER / GH ACTIONS / PROMETHEUS / VERCEL",
  steps: [
    { number: "01", title: "Commit", label: "Git push", text: "..." },
    // 02 Test, 03 Build, 04 Ship, 05 Observe
  ],
}
```

La section pipeline n'a pas d'entrée dans le menu principal : elle vit entre « Méthode » et « Parcours », avec l'ancre `#livraison`. Si vous voulez l'ajouter au menu, ajoutez une entrée dans `nav` avec `id: "livraison"`.

5.8 Parcours, formation, certifications

Tout est dans `about`. C'est le bloc le plus long après les projets. Il alimente la section « Parcours » et le prompt de l'assistant.

Textes de présentation

```
about: {
  lead: "Je m'appelle Zo Mahefa RANAIVO...",
  body1: "Formé à l'ENI, j'ai appris à relier le produit, le système et la qualité...",
  body2: "Je documente mes décisions, je teste ce que je livre...",
  targetKicker: "CE QUE JE RECHERCHE",
  targetItems: ["Web & mobile · backend", "DevOps & cloud", "Qualité & automatisation"],
  languagesLabel: "LANGUES",
  languages: "Français courant · Anglais technique... · Malgache natif",
  formatLabel: "FORMAT",
  format: "Remote friendly · réponse sous 48 h ouvrées",
}
```

Expériences

```
experiences: [
  {
    period: "2024 — Présent",
    title: "Développeur web & DevOps",
    company: "Freelance · Projets personnels",
    text: "Conception et développement d'applications métier...",
    tags: ["Spring Boot", "React", "Docker", "CI/CD"],
  },
  // ...
]
```

Chaque expérience affiche la période, le titre, l’entreprise, le texte et les tags. L’ordre du tableau est l’ordre d’affichage : le plus récent d’abord.

Formation

```
education: [
  {
    period: "2025 — 2026",
    title: "Master 1 en informatique",
    school: "ENI — École Nationale d'Informatique",
    detail: "Ingénierie des systèmes d'information. Formation en cours.",
  },
  // ...
]
```

Certifications

```
certifications: [
  {
    title: "Certification Spring Boot",
    issuer: "Mind Luster",
    year: "2026",
    image: "/certifcatSping-boot2.jpeg",
    href: "/certificatSpringBoot.pdf",
  },
  {
    title: "Certification UML",
    issuer: "Mind Luster",
    year: "2026",
    image: "/uml-certification.jpeg",
    href: "/uml-certification-zo.pdf",
  },
]
```

- `image` est la vignette affichée (JPEG/PNG dans `public/`).
- `href` est le PDF ouvert au clic, dans un nouvel onglet.
- Les noms de fichiers actuels contiennent une faute (`certifcatSping-boot2.jpeg`). Si vous les renommez, mettez à jour `image` et `href`.
- Pour ajouter une certification : déposez la vignette et le PDF dans `public/`, puis ajoutez un objet dans les tableaux FR et EN.

> **TAILLE DES VIGNETTES**
> Le composant `about.tsx` déclare `width={1600}` et `height={1131}` pour les images de certification. Utilisez de préférence un format paysage proche de ce ratio pour éviter les recadrages étranges. Les images sont optimisées par `next/image`.

Savoir-être

```
softSkills: [
  {
    title: "Résolution de problèmes",
    text: "Partir du symptôme, isoler la cause...",
    proof: "Diagnostics sur les pipelines, les environnements...",
  },
  // ...
]
```

Chaque qualité est reliée à une preuve concrète. C'est ce qui distingue ce bloc d'une liste de mots-clés. Si vous ajoutez une qualité, ajoutez une preuve vérifiable.

5.9 Témoignages

```
testimonials: {
  items: [
    {
      name: "Quincaillerie Boutique de l'Évolution",
      company: "Boutique de l'Évolution · Quincaillerie",
      phone: "034 16 118 50",
      phoneHref: "tel:+261341611850",
      quote: "Nous suivions nos stocks et nos ventes sur des cahiers...",
      context: "ERP — stocks, ventes & trésorerie",
    },
  ],
  note: "Témoignages recueillis auprès de clients...",
}
```

> **DONNÉES PERSONNELLES**
> Les témoignages contiennent des noms et des numéros de téléphone. Ne publiez que des personnes qui ont donné leur accord. Pour retirer un témoignage, supprimez l'objet dans `fr` et `en`. La clé React étant `testimonial.name`, deux témoignages avec le même nom provoqueraient un avertissement React.

5.10 Contact et formulaire

Le bloc `contact` contient les libellés de la colonne de gauche et tous les textes du formulaire. Les champs eux-mêmes sont dans `src/components/site/contact-form.tsx` ; les textes sont dans `contact.form`.

```
contact: {
  directLabel: "ÉCRIRE DIRECTEMENT",
  recruiterLabel: "RECRUTEUR",
  recruiterTitle: "Un poste, un stage ou une alternance",
  recruiterText: "CV, disponibilité et références de projets.",
  clientLabel: "CLIENT",
  clientTitle: "Un produit à concevoir ou à fiabiliser",
  clientText: "Un premier échange de 30 minutes, sans engagement.",
  locationNote: "remote possible",
  cv: "CV PDF",
  types: [
    "Site ou application web",
    "Application mobile",
    "DevOps / CI-CD",
    "Qualité & tests",
    "Recrutement",
    "Autre",
  ],
  form: {
    head: "BRIEF / CONTACT",
    response: "RÉPONSE < 48 H",
    name: "Nom complet",
    // ...
  },
}
```

- `types` alimente la liste déroulante « Type de besoin ». Le choix est préfixé au message enregistré.
- `form.consent` est le texte du consentement. Il est obligatoire pour envoyer.
- `form.errorSend` et `form.fallbackLink` gèrent le repli e-mail.
- Les sujets des deux liens recruteur/client sont dans `contact-section.tsx`, pas dans `content.ts` : cherchez « Opportunité — recrutement » et « Mission — cadrage de projet ».

5.11 Footer et micro-libellés

`footer` contient l'accroche, les libellés et les mentions. `ui` contient tous les petits textes d'interface : boutons, galerie, accessibilité, thème, langue, menu. Si un bouton affiche un texte inattendu, cherchez-le dans `ui`.

```
footer: {
  taglineLine1: "Ingénieur logiciel — web, mobile,",
  taglineLine2: "DevOps et testeur QA.",
  navLabel: "Navigation de pied de page",
  contactLabel: "CONTACT",
  cv: "CV PDF",
  backToTop: "Haut de page",
  rights: "Tous droits réservés.",
  built: "Conçu, développé et déployé avec Next.js.",
}
```

5.12 Textes de l'assistant IA

Deux endroits distincts :

1. `content.ts` → `ai` : les textes visibles du widget (titre, accueil, suggestions, erreurs, disclaimer).

2. `src/lib/assistant-prompt.ts` : les règles et le contexte envoyés au modèle (ton, périmètre, profil, projets, stack…).

> **MISE À JOUR AUTOMATIQUE DU CONTEXTE**
> Le prompt est construit à partir de `defaultContent` (le français). Si vous modifiez un projet, une expérience ou une certification dans `content.ts`, l'assistant apprendra l'information au prochain build, sans toucher à `assistant-prompt.ts`. Vous ne modifiez ce fichier que pour changer les *règles* ou le ton.

5.13 CV, photo et images

Besoin	Fichier	Où le référencer
Remplacer le CV	`public/cv-zomahefa-ranaivo-2026-10.pdf`	`site.cv`. Conservez le même nom pour éviter les liens morts, ou mettez à jour `site.cv`.
Remplacer le portrait	`public/photo-cv.png`	Utilisé par `hero.tsx` et le JSON-LD (`image`).
Image de partage LinkedIn / réseaux	`public/og-image.png`	Open Graph et Twitter dans `layout.tsx`. Format 1200×630.
Captures de projets	`public/projects/*.png`	`featuredProjects[].gallery[].src`.
Vignettes de certification	`public/*.jpeg`	`about.certifications[].image`.
Attestations PDF	`public/*.pdf`	`about.certifications[].href`.
Icône Apple	`public/apple-icon.png`	`layout.tsx` et `manifest.ts`.
Polices	`public/fonts/`	`globals.css` et préchargement dans `layout.tsx`.

> **NOMS DE FICHIERS**
> Les chemins sont sensibles à la casse sur Vercel (Linux). Utilisez des minuscules, sans espaces ni accents : `mon-projet-capture-01.png`. Un fichier nommé `Capture 01.PNG` fonctionnera peut-être en local sur macOS/Windows mais renverra 404 en production.

5.14 SEO, métadonnées et partage

Le titre et la description globaux

Dans `src/app/layout.tsx` :

```
const title = "Zo Mahefa RANAIVO — Développeur web, DevOps & QA";
const description = "Portfolio de Zo Mahefa RANAIVO, développeur web & mobile...";
```

Le template de titre ajoute automatiquement « — Zo Mahefa RANAIVO » aux pages qui définissent leur propre titre (comme l'admin).

Les autres champs à connaître

Champ	Effet
`metadataBase`	Base des URLs absolues pour Open Graph et Twitter. Dépend de `siteUrl`.
`keywords`	Mots-clés (peu utilisés par Google, utiles pour d'autres moteurs).

Champ	Effet
`alternates.canonical`	URL canonique de la page d'accueil.
`openGraph`	Titre, description, image 1200×630, locale `fr_FR`.
`twitter`	Carte `summary_large_image`.
`robots`	Indexation et directives Googlebot.
`verification.google`	Code de vérification Search Console. Le fichier HTML de vérification est aussi dans `public/`.
`schema` (JSON-LD)	Décrit la personne, le site et la page de profil pour les moteurs.

Changer de domaine

1. Ajoutez le domaine dans Vercel (Settings → Domains).
2. Configurez les enregistrements DNS demandés.
3. Mettez `NEXT_PUBLIC_SITE_URL` à la nouvelle URL, sans slash final.
4. Redéployez.
5. Vérifiez `/sitemap.xml`, `/robots.txt` et le code source (canonical, JSON-LD).
6. Demandez à Google Search Console de réindexer, et rafraîchissez l'aperçu LinkedIn.

Le JSON-LD

Le JSON-LD décrit `Person`, `WebSite` et `ProfilePage`. Il contient le nom, les rôles, la photo, l'e-mail, le téléphone, l'adresse, les réseaux, l'école, les langues et les compétences. Si vous changez de nom, de téléphone ou de réseau social, mettez à jour `site` **et** le JSON-LD de `layout.tsx` (certaines valeurs y sont encore écrites en dur).

5.15 Vérifier après chaque modification

- ☐ La même clé a été modifiée dans `fr` et dans `en`.
- ☐ `npm test` passe (parité FR/EN, aucun texte vide).
- ☐ `npm run build` passe (TypeScript, imports, routes).
- ☐ Le rendu local est correct en français et en anglais.
- ☐ Les liens ajoutés répondent (GitHub, démo, PDF).
- ☐ Les images ajoutées sont dans `public/` avec un nom en minuscules.
- ☐ Le texte ne contient pas de secret ni de donnée personnelle non autorisée.
- ☐ Le commit est poussé : Vercel redéploie automatiquement.

CHAPITRE 06

Ajouter un projet

Il existe deux niveaux de mise en avant : le **projet vedette**, qui occupe une grande ligne avec étude de cas et visuel, et le **projet d’archive**, qui tient sur une ligne compacte. Les deux se modifient dans `content.ts`, jamais en base.

6.1 Ajouter un projet vedette — pas à pas

Étape 1 — Préparer les images

Déposez les captures dans `public/projects/`. Utilisez des noms courts, en minuscules, sans espaces :

```
public/projects/
├── mon-projet-tableau-de-bord.png
├── mon-projet-liste.png
└── mon-projet-detail.png
```

- Format conseillé : PNG ou WebP, largeur 1 200–1 600 px.
- Évitez les captures avec des données personnelles réelles (noms, e-mails, numéros).
- Si vous n’avez pas de capture, utilisez un `visual` existant : `incident`, `erp` ou `pipeline`.

Étape 2 — Ajouter l’objet en français

Dans `const fr`, cherchez `featuredProjects: [`. Ajoutez votre objet à l’endroit où vous voulez qu’il apparaisse. Pour le mettre en premier, placez-le juste après le crochet ouvrant.

```
{
  id: "mon-projet",
  index: "04",
  title: "Mon projet",
  subtitle: "Gestion de stock · PME",
  year: "2026",
  category: "Web app / Fullstack",
  summary: "Une application pour suivre les stocks et les ventes en temps réel.",
  context: "Projet freelance réalisé pour une PME locale.",
  problem: "Les stocks étaient suivis sur tableur, avec des ruptures et des doublons.",
  role: "Cadrage, modélisation des données, API, interface et déploiement.",
  outcome: "Un tableau de bord unique, des alertes de seuil et une mise en production
reproductible.",
  stack: ["Next.js", "TypeScript", "PostgreSQL", "Docker"],
  visual: "erp",
  gallery: [
    {
      src: "/projects/mon-projet-tableau-de-bord.png",
      alt: "Tableau de bord du projet Mon projet avec les indicateurs de stock",
      label: "Tableau de bord",
    },
    {
      src: "/projects/mon-projet-liste.png",
      alt: "Liste des produits et seuils d'alerte",
      label: "Liste des produits",
    },
  ],
  repo: "https://github.com/Zomahefa/mon-projet",
  demo: "https://mon-projet.vercel.app/",
  demoLabel: "Voir la démo",
},
```

LES CHAMPS OBLIGATOIRES

`id`, `index`, `title`, `subtitle`, `year`, `category`, `summary`, `context`, `problem`, `role`, `outcome`, `stack` et `visual` sont requis par le type `FeaturedProject`. Un oubli fait échouer `npm run build` avec un message TypeScript explicite.

Étape 3 — Ajouter le même objet en anglais

Dans `const en`, cherchez à nouveau `featuredProjects: [`. Ajoutez le même objet, avec les textes traduits. Conservez strictement les mêmes `id`, `index`, `year`, `stack`, chemins d’images, liens et `visual`.

```
{
  id: "mon-projet",
  index: "04",
  title: "My project",
  subtitle: "Inventory management · SMB",
  year: "2026",
  category: "Web app / Fullstack",
  summary: "An application to track stock and sales in real time.",
  context: "Freelance project for a local SMB.",
  problem: "Inventory was tracked in spreadsheets, causing stockouts and duplicates.",
  role: "Scoping, data modeling, API, UI and deployment.",
  outcome: "A single dashboard, threshold alerts and a reproducible production release.",
  stack: ["Next.js", "TypeScript", "PostgreSQL", "Docker"],
  visual: "erp",
  gallery: [
    {
      src: "/projects/mon-projet-tableau-de-bord.png",
      alt: "Mon project dashboard with inventory indicators",
      label: "Dashboard",
    },
    {
      src: "/projects/mon-projet-liste.png",
      alt: "Product list and alert thresholds",
      label: "Product list",
    },
  ],
  repo: "https://github.com/Zomahefa/mon-projet",
  demo: "https://mon-projet.vercel.app/",
  demoLabel: "View demo",
},
```

Étape 4 — Ajuster les index et les compteurs

Si vous insérez le projet au milieu, renumérotez les `index` des projets suivants (`"01"`, `"02"`, `"03"`…) dans les deux langues. Pensez aussi aux statistiques du hero (`hero.stats`, « projets documentés ») et au `kicker` de l’archive si le nombre de projets change.

Étape 5 — Vérifier

```
npm test
npm run build
npm run dev
```

Ouvrez `http://localhost:3000`, vérifiez le projet en français, basculez en anglais, testez le carrousel au clavier (flèches gauche/droite), vérifiez les liens de démo et de code. Sur mobile, vérifiez que les images ne débordent pas.

6.2 Ajouter un projet d’archive

C’est plus simple : une ligne, pas d’image, pas d’étude de cas. Dans `archive.projects`, ajoutez :

```
{
  title: "Mon projet d'archive",
  category: "Desktop · JavaFX",
  year: "2026",
  stack: "JavaFX · SQLite",
  href: "https://github.com/Zomahefa/mon-projet-archive",
  label: "Code",
},
```

- `href` : si présent, la ligne est cliquable et affiche « Code ».
- `label` : affiché seulement si `href` est absent. Utilisez « Lab », « Labs » ou « Projet privé ».
- Mettez à jour `archive.kicker` (« ARCHIVE / 07 ») et, si besoin, le texte `copy`.

6.3 Cas particuliers

Projet sans dépôt public

N'ajoutez pas `repo`. Ajoutez `privateCode: true` : le lien « Étude de cas sur demande » apparaîtra et ouvrira un e-mail prérempli avec le nom du projet.

```
privateCode: true,
demo: "https://demo.example.com/",
demoLabel: "Voir la démo",
```

Projet sans capture d'écran

Utilisez un `visual` existant. Les trois schémas sont définis dans `src/components/site/project-visual.tsx` :

Valeur	Usage
`incident`	Signalement, supervision, incidents, workflow terrain.
`erp`	Tableaux de bord, gestion, stocks, ventes, trésorerie.
`pipeline`	CI/CD, tests, logs, alertes, infrastructure.

Créer un nouveau type de visuel

1. Dans `content.ts`, ajoutez la valeur à l'union :

   ```
   export type ProjectVisualKind = "incident" | "erp" | "pipeline" | "mobile";
   ```

2. Dans `src/components/site/project-visual.tsx`, ajoutez le rendu correspondant (SVG, blocs, textes).
3. Utilisez la nouvelle valeur dans vos objets FR et EN.

> **ALTERNATIVE PLUS SIMPLE**
> Si vous n'avez qu'une capture, utilisez `gallery` avec un seul élément : le carrousel s'affiche en mode « aperçu », sans autoplay. C'est souvent plus convaincant qu'un schéma générique.

Images externes

`next.config.ts` autorise les images locales (`localPatterns: [{ pathname: "/**" }]`) mais ne déclare aucun `remotePatterns`. Une URL externe dans `gallery[].src` sera refusée par `next/image`. Le plus

sûr est de télécharger l'image dans `public/projects/`. Sinon, ajoutez le domaine dans `next.config.ts` :

```
images: {
  // ...
  remotePatterns: [
    { protocol: "https", hostname: "images.example.com" },
  ],
},
```

6.4 Checklist d'ajout de projet

- ☐ Images déposées dans `public/projects/`, noms en minuscules.
- ☐ Objet ajouté dans `featuredProjects` côté `fr`.
- ☐ Objet identique ajouté côté `en` (mêmes id, index, stack, liens, images).
- ☐ Index des autres projets renumérotés si nécessaire.
- ☐ Statistiques du hero et `archive.kicker` à jour.
- ☐ `npm test` et `npm run build` passent.
- ☐ Vérification visuelle desktop + mobile + FR/EN.
- ☐ Liens de démo et de code testés.
- ☐ Commit et push : déploiement Vercel automatique.

CHAPITRE 07

Admin, base de données et sécurité

Le portfolio n’a pas de CMS. La base de données sert uniquement à recevoir les messages du formulaire et à compter les visites. L’administration est une page privée qui lit ces deux tables. Ce chapitre explique le schéma, les règles d’accès et les opérations courantes.

7.1 Les deux tables

`messages`

Colonne	Type	Rôle
`id`	TEXT PRIMARY KEY	UUID généré par `crypto.randomUUID()`.
`name`	TEXT NOT NULL	Nom du visiteur (max 120 caractères côté API).
`email`	TEXT NOT NULL	E-mail validé par regex (max 180).
`phone`	TEXT NOT NULL	Téléphone optionnel (max 30), chaîne vide si absent.
`message`	TEXT NOT NULL	Message (max 5 000), avec le type de besoin préfixé si fourni.
`created_at`	TEXT NOT NULL	Date ISO 8601, tri décroissant dans l’admin.

`page_views`

Colonne	Type	Rôle
`id`	TEXT PRIMARY KEY	UUID de la vue.
`path`	TEXT NOT NULL	Chemin visité, commence par `/`, max 200.
`referrer`	TEXT DEFAULT ''	Page d’origine, max 300.
`country`	TEXT DEFAULT ''	Code pays à deux lettres fourni par Vercel/Cloudflare.
`device`	TEXT DEFAULT ''	`Desktop`, `Mobile` ou `Tablet`.
`visitor_id`	TEXT DEFAULT ''	UUID local du navigateur, pour les visiteurs uniques.
`created_at`	TEXT NOT NULL	Date ISO 8601.

Trois index sont créés :

```
idx_messages_created_at      ON messages(created_at)
idx_page_views_created_at    ON page_views(created_at)
idx_page_views_visitor       ON page_views(visitor_id)
```

7.2 Turso ou SQLite local ?

`src/lib/db.ts` choisit automatiquement :

```
const isTurso = !!process.env.TURSO_DB_URL;

// Si TURSO_DB_URL est défini :
createClient({ url: TURSO_DB_URL, authToken: TURSO_DB_TOKEN })

// Sinon :
createClient({ url: "file:data/portfolio.db" })
```

- **En local**, sans variables Turso : un fichier `data/portfolio.db` est créé. Il est ignoré par Git.
- **En production**, avec `TURSO_DB_URL` et `TURSO_DB_TOKEN` : la base distante est utilisée.
- Les tables sont créées automatiquement au premier accès grâce à `ensureSchema()`, mais il est recommandé de lancer `npm run db:init` pour vérifier les droits et initialiser explicitement.

> **NE PAS MÉLANGER LES ENVIRONNEMENTS**
> Si vous lancez `npm run dev` avec les variables Turso de production dans `.env.local`, vous écrirez dans la vraie base pendant vos tests. Utilisez soit le SQLite local, soit une base Turso dédiée au développement.

7.3 Initialiser la base

```
npm run db:init
```

Le script `scripts/init-db.mjs` :

1. Charge `.env.local` via `@next/env`.
2. Choisit Turso si `TURSO_DB_URL` existe, sinon `data/portfolio.db`.
3. Exécute les `CREATE TABLE IF NOT EXISTS` et les index.
4. Affiche la cible, le nombre de messages, le nombre de vues et les tables.

```
✓ Base initialisée
  cible        : Turso
  messages     : 0
  pages vues   : 0
  tables       : messages, page_views
```

Le script est idempotent : vous pouvez le relancer sans perdre les données.

7.4 L'administration

URL : `/admin`. La page est en `noindex, nofollow` et n'est pas listée dans le sitemap. Elle n'est pas liée depuis le site public.

Connexion

1. Le formulaire envoie le mot de passe à `/api/admin/login`.
2. La route limite à 6 tentatives par 15 minutes et par IP.
3. Si `ADMIN_PASSWORD` n'est pas configuré : réponse 503 explicite.
4. La comparaison se fait en temps constant (XOR caractère par caractère).
5. En cas de succès, un cookie `zm_admin_session` est posé : `HttpOnly`, `SameSite=Strict`, `Secure` en production, durée 8 heures.

6. Le jeton est `expiration.signature`, signature HMAC-SHA256 avec `ADMIN_PASSWORD`.

Onglet Messages

- Liste les messages du plus récent au plus ancien.
- Recherche locale sur le nom, l'e-mail et le message.
- Affiche les coordonnées, le type de besoin et la date formatée en heure de Fianarantsoa.
- Suppression unitaire avec confirmation.

Onglet Audience

Indicateur	Définition
Vues totales	`COUNT(*)` sur `page_views`.
7 derniers jours	Vues dont `created_at` est dans les 7 derniers jours.
Aujourd'hui	Vues dont la date correspond à la date courante.
Visiteurs uniques	`COUNT(DISTINCT visitor_id)`, hors identifiants vides.
Activité 14 jours	Nombre de vues par jour, pour le graphique.
Top pages	5 chemins les plus visités.
Top pays	5 pays les plus fréquents (si disponible).
Top appareils	5 types d'appareils.
Visites récentes	25 dernières vues avec chemin, référent, pays, appareil, date.

7.5 Authentification et sécurité

Le mot de passe admin

- Variable `ADMIN_PASSWORD`, jamais dans Git.
- Minimum recommandé : 12 caractères, majuscules, minuscules, chiffres et symboles.
- `isWeakPassword()` avertit dans les logs si le mot de passe est faible, mais ne bloque pas le démarrage.
- Changer le mot de passe invalide toutes les sessions existantes (la signature change).

Les limites de débit

Route	Limite	Fenêtre
`/api/admin/login`	6 tentatives	15 minutes / IP
`/api/messages` (POST)	5 messages	10 minutes / IP
`/api/visits` (POST)	60 vues	10 minutes / IP
`/api/assistant` (POST)	25 questions	10 minutes / IP

> **LIMITE EN MÉMOIRE**
> Ces compteurs vivent dans une `Map` en mémoire, par instance serverless. Ils ne sont pas partagés entre toutes les instances Vercel et se réinitialisent à froid. C'est une protection simple contre les abus basiques, pas un pare-feu. Pour une protection forte, utilisez un service de rate limiting distribué (Upstash, Redis, Vercel Firewall…).

En-têtes de sécurité

Définis dans `next.config.ts` et appliqués en production :

En-tête	Valeur / effet
`Content-Security-Policy`	`default-src 'self'`, scripts et styles inline autorisés, images `self data: blob:`, `object-src 'none'`, `frame-ancestors 'none'`.
`X-Content-Type-Options`	`nosniff`.
`X-Frame-Options`	`DENY` (pas d'iframe).
`Referrer-Policy`	`strict-origin-when-cross-origin`.
`Permissions-Policy`	Caméra, micro, géolocalisation et paiement désactivés.
`Strict-Transport-Security`	HSTS 2 ans, sous-domaines, preload.
`Cross-Origin-Opener-Policy`	`same-origin`.

En développement, la CSP est retirée pour ne pas gêner le rechargement à chaud de Next.js. Les autres en-têtes restent.

7.6 Opérations courantes

Supprimer les messages de test

Le plus simple : connectez-vous à `/admin` et supprimez-les un par un. Pour repartir de zéro sur la base locale :

```
rm -f data/portfolio.db
npm run db:init
```

> **NE FAITES PAS CELA EN PRODUCTION**
> Ne supprimez jamais `data/portfolio.db` sur un serveur en production : cela effacerait les messages et les visites. Sur Turso, une suppression est explicite et doit être réfléchie.

Nettoyer les visites de test sur Turso

```
turso db shell portfolio-zomahefa "DELETE FROM page_views;"
```

À utiliser uniquement juste après la mise en ligne, pour repartir d'un compteur propre. Les messages se suppriment depuis l'admin.

Sauvegarder la base

```
turso db shell portfolio-zomahefa ".dump" > backup-portfolio.sql
```

Conservez ce fichier hors du dépôt Git. Pour restaurer, utilisez `turso db shell` avec le fichier SQL, ou recréez une base et rejouez les insertions.

Vérifier le contenu d'une table

```
turso db shell portfolio-zomahefa "SELECT COUNT(*) FROM messages;"
turso db shell portfolio-zomahefa "SELECT COUNT(*) FROM page_views;"
turso db shell portfolio-zomahefa "SELECT name FROM sqlite_master WHERE type='table';"
```

7.7 Vie privée et données

- Les messages contiennent des données personnelles (nom, e-mail, téléphone). Ne les exportez pas dans un dépôt public.
- L’admin n’est pas indexé, mais il est accessible à toute personne qui connaît l’URL et le mot de passe.
- Les visites ne stockent pas l’IP en base : elle sert uniquement au rate limiting en mémoire.
- Le pays et l’appareil sont des données agrégées, pas des identifiants individuels.
- Prévoyez une politique de conservation : supprimez les messages traités après un délai raisonnable.

CHAPITRE 08

Assistant IA

L’assistant est un widget flottant présent sur toutes les pages publiques, sauf l’administration. Il répond aux questions sur le parcours, les projets, les compétences et la disponibilité. Il ne voit que des informations publiques et ne stocke aucune conversation en base.

8.1 Le parcours d’une question



8.2 Les variables d’environnement

Variable	Obligatoire	Rôle
`GROQ_API_KEY`	Oui si Groq	Clé API Groq. Utilisée par défaut si présente.
`GOOGLE_AI_API_KEY`	Oui si Gemini	Clé Google AI Studio.
`AI_PROVIDER`	Non	`groq` ou `google`. Force le fournisseur si la clé correspondante existe.
`GROQ_MODEL`	Non	Modèle Groq. Défaut : `openai/gpt-oss-120b`.
`GOOGLE_AI_MODEL`	Non	Modèle Gemini. Défaut : `gemini-2.5-flash`.

Ordre de résolution du fournisseur

1. Si `AI_PROVIDER=google` et `GOOGLE_AI_API_KEY` existe → Google.
2. Si `AI_PROVIDER=groq` et `GROQ_API_KEY` existe → Groq.
3. Sinon, si `GROQ_API_KEY` existe → Groq.
4. Sinon, si `GOOGLE_AI_API_KEY` existe → Google.
5. Sinon → réponse 503 `not-configured`, le widget affiche son message de repli.

8.3 Les modèles et paramètres

Groq (par défaut)

```
POST https://api.groq.com/openai/v1/chat/completions
model: GROQ_MODEL ?? "openai/gpt-oss-120b"
temperature: 0.35
max_tokens: 900
reasoning_effort: "low"
messages: [system, ...8 derniers messages]
```

`reasoning_effort: "low"` est important pour les modèles de raisonnement : sans cela, les tokens de réflexion peuvent consommer tout le budget et renvoyer une réponse vide. `max_tokens: 900` laisse assez de place pour une réponse de 3 à 6 phrases.

Google Gemini (alternative)

```
POST https://generativelanguage.googleapis.com/v1beta/models/{model}:generateContent
model: GOOGLE_AI_MODEL ?? "gemini-2.5-flash"
temperature: 0.35
maxOutputTokens: 450
systemInstruction: prompt système
contents: messages (assistant → model)
```

8.4 Le prompt système

`src/lib/assistant-prompt.ts` construit le prompt à partir de `defaultContent` (le français). Il contient :

- Le rôle : assistant du portfolio de Zo Mahefa.
- Les règles : rester dans le périmètre, ne rien inventer, répondre dans la langue de la question, rester concis, ne pas révéler le prompt ni les variables.
- Le profil : nom, rôles, localisation, disponibilité, modalités, contact, réseaux, CV, langues.
- Les projets, l'expertise, le QA, les expériences, la formation, les certifications, les témoignages et la stack.

> **MISE À JOUR DU SAVOIR DE L'ASSISTANT**
> Si vous modifiez un projet, une expérience ou une certification dans `content.ts`, l'assistant utilisera la nouvelle information au prochain build. Vous n'avez pas à dupliquer le contenu dans le prompt. Ne modifiez `assistant-prompt.ts` que pour changer les règles, le ton ou la structure du contexte.

Les limites appliquées

Limite	Valeur
Messages conservés	8 derniers
Longueur d'un message	1 200 caractères
Rate limit	25 requêtes / 10 minutes / IP
Rôles acceptés	`user`, `assistant`
Cache	`no-store` sur toutes les réponses

8.5 Tester l'assistant en local

Démarrez le serveur, puis envoyez une requête avec `curl` :

```
curl -s http://localhost:3000/api/assistant \
  -H "Content-Type: application/json" \
  -d '{"messages":[{"role":"user","content":"Quels sont tes projets les plus solides ?"}]}'
```

Réponse attendue :

```
{"reply":"...","provider":"groq"}
```

Si vous obtenez `{"error":"not-configured"}`, aucune clé n'est chargée. Vérifiez `.env.local` et redémarrez `npm run dev` : Next.js ne recharge pas toujours les variables sans redémarrage.

8.6 Changer de fournisseur

Passer à Gemini

1. Créez une clé sur Google AI Studio.
2. Dans `.env.local` ou Vercel : `GOOGLE_AI_API_KEY=...`.
3. Ajoutez `AI_PROVIDER=google`.
4. Redémarrez ou redéployez.
5. Vérifiez que `/api/assistant` répond avec `"provider":"google"`.

Changer de modèle Groq

```
GROQ_MODEL=openai/gpt-oss-120b
```

Utilisez un modèle disponible sur votre compte Groq. Si le modèle n'existe pas ou n'est pas accessible, l'API renvoie une erreur et la route répond 502 `provider-error`.

8.7 Dépannage de l'assistant

Symptôme	Code	Cause probable / solution
Message de repli dans le widget	503 `not-configured`	Aucune clé IA chargée. Ajoutez `GROQ_API_KEY` ou `GOOGLE_AI_API_KEY`, puis redéployez.
« Réessayez dans un instant »	429 `rate-limited`	25 requêtes en 10 minutes dépassées. Attendez ou testez depuis une autre IP.
Réponse vide / erreur	502 `provider-error`	Clé invalide, quota, modèle inexistant, panne du fournisseur. Vérifiez les logs Vercel et le nom du modèle.
Requête refusée	400 `empty`	Le tableau `messages` est vide ou ne contient aucun rôle valide.
Le widget ne s'ouvre pas	—	Vérifiez que `AiAssistant` est bien monté dans `layout.tsx` et que le chemin n'est pas `/admin`.
Réponse hors sujet	—	Le prompt système ou le contenu public contient une information ambiguë. Renforcez les règles dans `assistant-prompt.ts`.

8.8 Coût, confidentialité et limites

- Chaque question consomme des tokens chez le fournisseur. Le paramètre `max_tokens` limite la dépense, mais ne la supprime pas.
- Aucune conversation n'est enregistrée en base. L'historique vit dans l'état React du widget et disparaît au rechargement.
- Le prompt ne contient que des informations publiques. Il ne faut jamais y ajouter de données personnelles, de secrets ou de messages reçus.
- Le modèle peut se tromper. Le widget affiche un disclaimer et propose d'écrire directement.
- Les réponses sont générées à la volée : elles ne sont pas validées par un humain avant affichage.

CHAPITRE 09

Qualité, tests et sécurité

Avant chaque déploiement, quatre commandes suffisent à éviter la majorité des régressions : tests, lint, vérification TypeScript et build. Ce chapitre explique ce que chacune vérifie et ce qu'elle ne vérifie pas.

9.1 Les commandes

Commande	Ce qu'elle fait
`npm test`	Lance Vitest une fois. Vérifie la parité FR/EN, l'authentification et les regex de validation.
`npm run test:watch`	Relance les tests à chaque modification.
`npm run lint`	ESLint avec les règles Next.js et TypeScript.
`npx tsc --noEmit`	Vérifie les types sans générer de fichiers. (Le build le fait aussi.)
`npm run build`	Compile le site, vérifie les types, génère les routes et les pages statiques.
`npm run start`	Démarre le build de production en local (après `npm run build`).

9.2 Les tests présents

Fichier	Ce qu'il vérifie
`src/lib/content.test.ts`	La structure de `fr` et `en` est identique (mêmes clés, mêmes longueurs de tableaux) et aucun texte n'est vide. C'est le test le plus important pour éviter les oublis de traduction.
`src/lib/auth.test.ts`	Le mot de passe correct est accepté ; les mauvais formats sont refusés ; la détection de mot de passe faible fonctionne.
`src/lib/validation.test.ts`	Les regex e-mail et téléphone définies dans le test. Il n'importe pas de module `validation.ts` : c'est un test de référence, pas la validation utilisée en production.
`src/components/language-provider.test.tsx`	Le provider de langue et la bascule FR/EN.

> **CE QUE LES TESTS NE COUVRENT PAS**
>
> Il n'y a pas de test end-to-end du formulaire, de l'admin ou de l'assistant. Ces parcours doivent être vérifiés manuellement après déploiement (voir 11.9). Les tests ne remplacent pas un clic réel sur le site.

9.3 Ce que vérifie le build

- Les imports et les alias `@/`.
- Les types de `content.ts` : un champ manquant dans un projet, une certification ou un témoignage fait échouer la compilation.
- Les composants client/serveur : un hook utilisé dans un composant serveur provoque une erreur.

- Les routes : pages, API, sitemap, robots, manifest.
- Les images locales référencées.

9.4 Vérifications manuelles avant mise en ligne

- ☐ La page d'accueil se charge en français et en anglais.
- ☐ Le thème clair/sombre fonctionne et persiste après rechargement.
- ☐ Le menu mobile s'ouvre, se ferme avec Échap et verrouille le scroll.
- ☐ Les liens d'ancrage du header et du footer arrivent sur les bonnes sections.
- ☐ Les projets vedettes affichent leurs images et leurs liens.
- ☐ Le carrousel répond aux flèches et se met en pause au survol.
- ☐ Le formulaire envoie un message et affiche l'écran de succès.
- ☐ Le message apparaît dans `/admin`, puis peut être supprimé.
- ☐ Une visite apparaît dans l'onglet Audience (hors `/admin`).
- ☐ L'assistant répond à une question simple.
- ☐ Les PDF de certification s'ouvrent.
- ☐ Le CV se télécharge.
- ☐ Les pages `/projets`, `/contact`, `/competences` redirigent correctement.
- ☐ `/sitemap.xml` et `/robots.txt` sont accessibles.

9.5 Accessibilité et préférences

- Un lien « Aller au contenu » est présent au début du document.
- Les animations respectent `prefers-reduced-motion` : compteurs, machine à écrire, galerie et révélations.
- Le menu mobile gère Échap et le focus.
- Les images ont des textes alternatifs ; les icônes décoratives sont masquées aux lecteurs d'écran.
- Les contrastes du thème sombre sont définis dans `globals.css`.
- Les formulaires ont des libellés associés et des messages d'erreur explicites.

9.6 Sécurité : la checklist

- ☐ `.env.local` n'est jamais commité.
- ☐ `ADMIN_PASSWORD` fait au moins 12 caractères et n'est pas un mot courant.
- ☐ Le jeton Turso est stocké uniquement dans Vercel et `.env.local`.
- ☐ La clé Groq/Gemini n'est jamais exposée côté client (pas de préfixe `NEXT_PUBLIC_`).
- ☐ Les dépendances sont à jour : `npm audit --omit=dev`.
- ☐ Les en-têtes de sécurité sont actifs en production.
- ☐ L'admin est en `noindex` et n'est pas lié depuis le site.
- ☐ Les messages reçus ne sont pas exportés dans un dépôt public.
- ☐ Le CV et les attestations ne contiennent pas d'informations que vous ne voulez pas publier.

NE JAMAIS FAIRE

Ne mettez jamais `ADMIN_PASSWORD`, `TURSO_DB_TOKEN`, `GROQ_API_KEY` ou `GOOGLE_AI_API_KEY` dans un fichier versionné, une capture d’écran, un ticket ou ce PDF. Si un secret a fuité, révoquez-le et recréez-le immédiatement.

CHAPITRE 10

Développement local

10.1 Prérequis

- Node.js 20 ou 22 (le projet utilise Next.js 16).
- npm (fourni avec Node).
- Git.
- Un éditeur (VS Code, Cursor…).

10.2 Installation

```
cd /chemin/vers/le/portfolio
npm install
```

Utilisez `npm ci` si vous voulez reproduire exactement les versions du `package-lock.json` (CI, machine neuve).

10.3 Fichier d’environnement local

Copiez le modèle et remplissez les valeurs :

```
cp .env.example .env.local
```

```
# Site
NEXT_PUBLIC_SITE_URL=http://localhost:3000

# Admin
ADMIN_PASSWORD=un-mot-de-passe-fort-de-12-caracteres-minimum

# Base (laisser vide pour SQLite local)
TURSO_DB_URL=
TURSO_DB_TOKEN=

# Assistant IA
AI_PROVIDER=groq
GROQ_API_KEY=
GOOGLE_AI_API_KEY=
```

> **VALEURS FACTICES**
> Les valeurs ci-dessus sont des exemples. Ne recopiez pas un vrai mot de passe ou une vraie clé dans un document, un chat ou un commit.

10.4 Initialiser la base locale

```
npm run db:init
```

Sans `TURSO_DB_URL`, cela crée `data/portfolio.db`. Le dossier `data/` est ignoré par Git.

10.5 Lancer le serveur

```
npm run dev
```

Ouvrez `http://localhost:3000`. Pour tester sur un autre port :

```
npm run dev -- --port 3100
```

10.6 Tester le build de production en local

```
npm run build
npm run start
```

Le build de production est plus strict que le mode développement : c'est celui qui doit passer avant un déploiement.

10.7 Docker (optionnel)

Le dépôt contient un `Dockerfile` et un script `run.sh` pour un déploiement auto-hébergé. Ce n'est pas la voie utilisée par Vercel.

```
docker build -t zomahefaranaivo/portfolio .
./run.sh latest
```

Le script monte `./data` dans `/app/data` et charge `.env.local`. Sans variables Turso, la base SQLite persiste dans ce volume.

10.8 Problèmes locaux fréquents

Symptôme	Solution
Port 3000 déjà utilisé	`npm run dev -- --port 3100` ou arrêtez l'autre processus.
Variables non prises en compte	Redémarrez `npm run dev` : les variables serveur ne sont pas toujours rechargées à chaud.
Base introuvable	Lancez `npm run db:init`. Vérifiez que `data/` est accessible en écriture.
Erreur de module	Supprimez `node_modules` et `.next`, puis `npm ci`.
Tests qui échouent après une modification de contenu	Vous avez probablement oublié une clé en français ou en anglais. Lisez le chemin indiqué par `content.test.ts`.
Image 404	Vérifiez le nom exact dans `public/` et la casse.
Assistant 503	Ajoutez une clé IA dans `.env.local` et redémarrez.

CHAPITRE 11

Déploiement Vercel + Turso

Ce chapitre est la procédure complète, dans l'ordre, pour passer du dépôt local à un site public avec base de données distante. Comptez 20 à 30 minutes la première fois. Ne sautez pas l'étape d'initialisation de la base : c'est la cause n°1 des formulaires qui renvoient une erreur.

11.1 Ce que vous allez créer



11.2 Prérequis

- ☐ Un compte GitHub.
- ☐ Un compte Vercel (gratuit suffit pour commencer).
- ☐ Un compte Turso (gratuit suffit pour commencer).
- ☐ Une clé API Groq ou Google AI Studio pour l'assistant.
- ☐ Node.js 20+ et npm installés localement.
- ☐ Le projet qui build en local.
- ☐ Un nom de domaine, si vous ne voulez pas rester sur `*.vercel.app`.

11.3 Vérifications avant de déployer

Ne déployez jamais un état qui ne passe pas localement. Exécutez, dans l'ordre :

```
npm ci
npm run lint
npm test
npm run build
```

Le build doit afficher les routes générées, dont `/`, `/admin`, `/api/*`, `/sitemap.xml`, `/robots.txt` et `/manifest.webmanifest`. S'il échoue, corrigez d'abord : Vercel échouera de la même manière.

> **TEST DE PRODUCTION LOCAL**
> Après `npm run build`, lancez `npm run start` et testez le formulaire, l'admin et l'assistant sur `http://localhost:3000`. C'est le meilleur moyen de détecter un problème de variables avant de déployer.

11.4 Pousser le code sur GitHub

```
git status
git add .
git commit -m "chore: préparation du déploiement"
git remote -v
git push origin main
```

Si le dépôt n’existe pas encore, créez-le sur GitHub (privé ou public), puis :

```
git remote add origin https://github.com/<votre-compte>/<votre-depot>.git
git branch -M main
git push -u origin main
```

> **AVANT LE PREMIER PUSH**
> Vérifiez que `.env.local`, `data/` et les fichiers `*.db` ne sont pas suivis. `git status` ne doit pas les afficher. Si un secret a déjà été commité, considérez-le comme compromis et changez-le.

11.5 Créer la base Turso

Option A — Via le tableau de bord (recommandé pour débuter)

1. Créez un compte sur `turso.tech`.
2. Dans le dashboard, créez une base de données, par exemple `portfolio-zomahefa`.
3. Choisissez une région proche de vos visiteurs (par exemple Francfort pour l’Europe/Afrique de l’Est, ou la région proposée la plus proche).
4. Copiez l’URL de connexion : elle commence par `libsql://`.
5. Créez un jeton d’accès (token) pour cette base. Copiez-le immédiatement : il n’est affiché qu’une fois.

Option B — Via la CLI Turso

```
# Installation (macOS / Linux)
curl -sSfL https://get.tur.so/install.sh | bash

# ou avec Homebrew
brew install tursodatabase/tap/turso

# Connexion
turso auth signup
# ou
turso auth login

# Création de la base (exemple de région : fra)
turso db create portfolio-zomahefa --location fra

# Récupérer l'URL libsql://
turso db show portfolio-zomahefa --url

# Créer un jeton
turso db tokens create portfolio-zomahefa

# Vérifier
turso db list
```

> **DEUX VALEURS À CONSERVER**
>
> **URL** : `libsql://portfolio-zomahefa-<organisation>.turso.io`
>
> **Token** : une longue chaîne secrète.
>
> Ne confondez pas avec l'URL HTTPS du dashboard. Le client libSQL attend le schéma `libsql://`.

11.6 Initialiser le schéma sur Turso

Vous pouvez le faire depuis votre machine, avant même de déployer Vercel. Cela vérifie que l'URL et le jeton fonctionnent.

```
export TURSO_DB_URL="libsql://portfolio-zomahefa-...turso.io"
export TURSO_DB_TOKEN="le-jeton-secret"
npm run db:init
```

Sur Windows PowerShell :

```
$env:TURSO_DB_URL="libsql://..."
$env:TURSO_DB_TOKEN="..."
npm run db:init
```

Résultat attendu :

```
✓ Base initialisée
  cible        : Turso
  messages     : 0
  pages vues   : 0
  tables       : messages, page_views
```

Si vous obtenez une erreur d'authentification, le jeton est faux ou a été révoqué. S'il manque des droits, recréez un jeton en lecture/écriture sur cette base.

11.7 Importer le projet dans Vercel

1. Connectez-vous à `vercel.com`.
2. Cliquez sur **Add New → Project**.
3. Importez le dépôt GitHub qui contient le portfolio.
4. Vérifiez les réglages :
   - **Framework Preset** : Next.js (détecté automatiquement).
   - **Root Directory** : `./` (ou le sous-dossier si monorepo).
   - **Build Command** : `npm run build`.
   - **Install Command** : `npm install` (ou `npm ci`).
   - **Output Directory** : laisser vide (Next.js gère).
   - **Node.js Version** : 20.x ou 22.x.
5. Ouvrez **Environment Variables** et ajoutez les variables de la section suivante.
6. Cliquez sur **Deploy**.

11.8 Les variables d'environnement Vercel

Variable	Exemple	Rôle
`NEXT_PUBLIC_SITE_URL`	`https://zo-mahefa-it.vercel.app`	URL publique, sans slash final. Sert au canonical, au sitemap, aux robots, à l'Open Graph et au JSON-LD.
`ADMIN_PASSWORD`	*mot de passe fort*	Accès à `/admin`. 12 caractères minimum, majuscules, chiffres, symboles.
`TURSO_DB_URL`	`libsql://...`	URL de la base Turso.
`TURSO_DB_TOKEN`	*jeton secret*	Jeton d'accès Turso.
`GROQ_API_KEY`	*clé Groq*	Assistant IA via Groq.
`AI_PROVIDER`	`groq`	Optionnel. Force `groq` ou `google`.
`GOOGLE_AI_API_KEY`	*clé Google*	Optionnel si vous utilisez Gemini.
`GROQ_MODEL`	`openai/gpt-oss-120b`	Optionnel. Surcharge le modèle Groq.
`GOOGLE_AI_MODEL`	`gemini-2.5-flash`	Optionnel. Surcharge le modèle Gemini.

PORTÉE DES VARIABLES

Ajoutez au minimum les variables à l'environnement **Production**. Si vous voulez que les Preview Deployments fonctionnent aussi, ajoutez-les à **Preview** — mais utilisez alors une base Turso de test, pas la base de production. Sinon, chaque preview écrira de vrais messages et de vraies visites.

POURQUOI `NEXT_PUBLIC_` ?

Ce préfixe rend la variable lisible côté navigateur. Ici, ce n'est pas un secret : c'est simplement l'URL publique du site. Ne mettez jamais une clé API ou un mot de passe derrière un préfixe `NEXT_PUBLIC_`.

11.9 Déployer

1. Cliquez sur **Deploy**. Vercel installe les dépendances, lance `npm run build`, puis publie.
2. Suivez les logs de build. Une erreur TypeScript ou un module manquant apparaît ici.
3. À la fin, Vercel fournit une URL du type `https://portfolio-xxxx.vercel.app`.
4. Ouvrez cette URL. Le site doit s'afficher même avant d'ajouter un domaine personnalisé.
5. Si `NEXT_PUBLIC_SITE_URL` pointe déjà vers le domaine final, les métadonnées seront correctes. Sinon, mettez-la à jour et redéployez.

Variante CLI Vercel (optionnelle)

```
npm i -g vercel
vercel login
vercel
vercel --prod
```

La CLI guide la création du projet et permet d'ajouter des variables :

```
vercel env add ADMIN_PASSWORD production
vercel env add TURSO_DB_URL production
vercel env add TURSO_DB_TOKEN production
vercel env add GROQ_API_KEY production
vercel env add NEXT_PUBLIC_SITE_URL production
```

Après un changement de variable, relancez `vercel --prod` ou cliquez sur **Redeploy** dans le dashboard.

11.10 Vérifier le déploiement

1. La page et les en-têtes

```
curl -I https://votre-domaine.tld
curl -I https://votre-domaine.tld/admin
curl -s https://votre-domaine.tld/robots.txt
curl -s https://votre-domaine.tld/sitemap.xml
```

- La page d'accueil répond 200.
- `/admin` répond 200 mais contient `noindex`.
- `robots.txt` exclut `/admin/` et `/api/`.
- `sitemap.xml` contient l'URL finale, pas `localhost`.
- Les en-têtes `content-security-policy`, `strict-transport-security`, `x-frame-options` sont présents.

2. Le contenu

- Le hero affiche le bon nom, les bons rôles, les bonnes statistiques.
- Les projets vedettes et l'archive sont complets.
- Les images se chargent (pas de 404 dans la console).
- Le CV et les PDF de certification s'ouvrent.
- La bascule FR/EN fonctionne et persiste après rechargement.

3. Le formulaire et l'admin

1. Envoyez un message de test avec une vraie adresse e-mail.
2. Vérifiez l'écran de succès.
3. Connectez-vous à `/admin` avec `ADMIN_PASSWORD`.
4. Le message doit apparaître dans l'onglet Messages.
5. Supprimez-le après vérification.

> **SI LE FORMULAIRE ÉCHOUE**
> Le formulaire affiche un lien `mailto:` de repli. Cela signifie que l'API n'a pas pu écrire en base. Vérifiez `TURSO_DB_URL`, `TURSO_DB_TOKEN`, puis les logs Vercel (Functions → Logs). Lancez `npm run db:init` avec les mêmes valeurs pour isoler le problème.

4. Les visites

1. Ouvrez le site dans un navigateur normal (pas en navigation privée avec bloqueur).
2. Naviguez entre les ancres ou rechargez la page.
3. Dans `/admin` → Audience, vérifiez que le total augmente.

Les robots, `/admin` et les requêtes trop fréquentes ne sont pas comptés. Si le compteur reste à zéro, vérifiez que `/api/visits` répond 204 dans l'onglet Réseau et que la base est accessible.

5. L'assistant

```
curl -s https://votre-domaine.tld/api/assistant \
  -H "Content-Type: application/json" \
  -d '{"messages":[{"role":"user","content":"Quels sont tes projets ?"}]}'
```

Attendu : `{"reply":"...","provider":"groq"}`. Si vous obtenez `not-configured`, la clé n'est pas dans l'environnement Production ou le déploiement n'a pas été refait après l'ajout.

6. Le partage social et le SEO

- Ouvrez `https://www.linkedin.com/post-inspector/` et testez l'URL.
- Vérifiez que l'image `og-image.png` apparaît (1200×630).
- Dans Google Search Console, ajoutez la propriété et soumettez le sitemap.
- Testez `https://votre-domaine.tld/manifest.webmanifest`.

11.11 Ajouter un domaine personnalisé

1. Dans Vercel : **Project → Settings → Domains**.
2. Ajoutez `votre-domaine.tld` et, si besoin, `www.votre-domaine.tld`.
3. Vercel affiche les enregistrements DNS à créer chez votre registrar :
   - `A` vers l'IP Vercel pour le domaine racine, ou
   - `CNAME` vers `cname.vercel-dns.com` pour un sous-domaine.
4. Attendez la propagation DNS (quelques minutes à quelques heures).
5. Mettez `NEXT_PUBLIC_SITE_URL` à `https://votre-domaine.tld` (sans slash final).
6. **Redéployez** : les métadonnées sont générées au build.

7. Vérifiez le canonical, le sitemap, le robots et le JSON-LD.

> **DEUX DOMAINES, UN SEUL CANONICAL**
> Si `zo-mahefa-it.vercel.app` et votre domaine personnalisé répondent tous les deux, choisissez le domaine final comme `NEXT_PUBLIC_SITE_URL` et redirigez l'autre vers celui-ci (Vercel permet de définir un domaine principal). Sinon, les moteurs voient deux URLs pour le même contenu.

11.12 Mettre à jour le site

Une fois le projet connecté à GitHub, le flux est automatique :

1. Modifiez le contenu ou le code en local.
2. Lancez `npm test` et `npm run build`.
3. Commit et push sur la branche `main`.
4. Vercel détecte le push et lance un nouveau déploiement.
5. Vérifiez le déploiement dans l'onglet **Deployments**.

```
git add .
git commit -m "content: mise à jour du projet X"
git push origin main
```

Pour les **variables d'environnement**, le flux est différent : modifiez-les dans Vercel, puis cliquez sur **Redeploy**. Un simple push ne suffit pas toujours à recharger une variable modifiée.

11.13 Revenir à une version précédente

1. Vercel → **Deployments**.
2. Repérez le dernier déploiement qui fonctionnait.
3. Menu **…** → **Promote to Production** (ou **Rollback** selon l'interface).
4. Vérifiez le site.
5. Corrigez le code en local, puis poussez un nouveau commit.

Le rollback Vercel est immédiat mais ne corrige pas Git : la prochaine publication reprendra le dernier commit. Pour un retour propre, faites aussi un `git revert`.

11.14 Dépannage complet

Symptôme	Cause probable	Solution
Le build échoue : `Module not found`	Import incorrect, casse différente, dépendance absente.	Vérifiez le chemin exact, relancez `npm ci`, comparez avec le build local.
Le build échoue : `Type error`	Champ manquant dans `content.ts` ou type incorrect.	Lisez le message : il indique la clé et le fichier. Corrigez le contenu, pas le type.
Le site affiche une ancienne version	Déploiement non déclenché, cache, mauvais projet Vercel.	Vérifiez le commit dans Deployments, forcez un redeploy, videz le cache du navigateur.

Symptôme	Cause probable	Solution
Formulaire : 503	Turso inaccessible, variables absentes, jeton invalide.	Vérifiez les variables Production, les logs Functions, puis `npm run db:init` avec les mêmes valeurs.
Admin : 503 « ADMIN_PASSWORD n'est pas configuré »	Variable absente de l'environnement Production.	Ajoutez-la dans Vercel, puis redéployez.
Admin : 401 après connexion	Cookie refusé, mot de passe changé, domaine différent, HTTP au lieu de HTTPS.	Utilisez HTTPS, videz les cookies du domaine, reconnectez-vous. Changer le mot de passe invalide les sessions.
Admin : « Trop de tentatives »	Rate limit 6/15 min atteint.	Attendez 15 minutes ou changez d'IP. Vérifiez que vous n'avez pas un script qui boucle.
Turso : `URL_SCHEME_NOT_SUPPORTED`	URL `https://` au lieu de `libsql://`.	Copiez l'URL libSQL dans le dashboard Turso.
Turso : `AUTH_TOKEN` / 401	Jeton révoqué, mauvais jeton, jeton d'une autre base.	Créez un nouveau jeton pour cette base, mettez-le dans Vercel, redéployez.
Turso : base vide / tables absentes	`db:init` jamais lancé et premier accès échoue.	Lancez `npm run db:init` avec les variables de production.
Assistant : 503 `not-configured`	Aucune clé IA.	Ajoutez `GROQ_API_KEY` ou `GOOGLE_AI_API_KEY`, redéployez.
Assistant : 502 `provider-error`	Clé invalide, quota, modèle inexistant.	Vérifiez les logs, le nom du modèle, le crédit du compte fournisseur.
Assistant : 429	25 requêtes / 10 min dépassées.	Attendez. En production, surveillez les abus.
Visites non comptées	Bloqueur, robot, `/admin`, rate limit, base indisponible.	Testez en navigation normale, vérifiez `/api/visits` (204), puis la base.
Images 404	Nom/casse différents, fichier absent de `public/`.	Vérifiez le chemin exact ; sur Vercel, tout est sensible à la casse.
Polices non chargées	Fichier absent ou CSP.	Vérifiez `public/fonts/` et l'en-tête `font-src 'self' data:`.
Open Graph vide sur LinkedIn	URL non absolue, image inaccessible, cache LinkedIn.	Vérifiez `og:image` dans le HTML, utilisez Post Inspector pour rafraîchir.
Canonical pointe vers l'ancien domaine	`NEXT_PUBLIC_SITE_URL` non mise à jour ou pas de redéploiement.	Modifiez la variable, redéployez, vérifiez le HTML source.
Le formulaire accepte mais rien n'arrive	Écriture dans une autre base (preview vs production).	Vérifiez la portée des variables et l'URL Turso utilisée par le déploiement.
Domaine : `DNS_PROBE_FINISHED_NXDOMAIN`	DNS pas propagé ou mal configuré.	Vérifiez les enregistrements A/CNAME chez le registrar, attendez la propagation.
Erreur de version Node	Version trop ancienne.	Réglez Node 20.x ou 22.x dans Vercel (Settings → General).

11.15 Sauvegardes, rotation et coûts

Sauvegarde

```
turso db shell portfolio-zomahefa ".dump" > backup-portfolio-$(date +%F).sql
```

Téléchargez aussi une copie depuis le dashboard Turso si l'option est disponible. Conservez les sauvegardes hors du dépôt Git.

Rotation d'un secret

1. Créez un nouveau jeton Turso (ou une nouvelle clé IA).
2. Mettez-le dans Vercel → Environment Variables.
3. Redéployez et testez.
4. Révoquez l'ancien jeton dans Turso (dashboard ou CLI).

Pour `ADMIN_PASSWORD`, changez la valeur dans Vercel et redéployez. Toutes les sessions existantes seront invalidées.

Coûts et limites

- Vercel et Turso proposent des paliers gratuits ; les limites et tarifs évoluent. Consultez les pages actuelles avant de dimensionner.
- L'assistant IA est le poste de coût variable principal : chaque question consomme des tokens.
- Les fonctions serverless ont une durée d'exécution limitée ; les appels IA doivent rester rapides.
- Surveillez le nombre de messages et de visites : la base peut grossir, mais reste très légère pour un portfolio.

11.16 Checklist de mise en production

- ☐ Build local vert (`npm run build`).
- ☐ Tests verts (`npm test`).
- ☐ Dépôt GitHub à jour, sans secret.
- ☐ Base Turso créée, URL et jeton récupérés.
- ☐ `npm run db:init` exécuté sur Turso.
- ☐ Projet Vercel importé.
- ☐ Toutes les variables Production ajoutées.
- ☐ Déploiement réussi.
- ☐ Page d'accueil, admin, formulaire, visites et assistant testés.
- ☐ Domaine ajouté et `NEXT_PUBLIC_SITE_URL` mis à jour.
- ☐ Redéploiement après changement d'URL.
- ☐ Sitemap, robots, canonical et Open Graph vérifiés.
- ☐ Message de test supprimé, visites de test nettoyées si besoin.

CHAPITRE 12

Publication LinkedIn

Voici un texte prêt à copier. Il présente le portfolio sans exagérer, en mettant en avant les trois projets, la double compétence produit/infrastructure/qualité et la recherche de stage, d'alternance ou de mission. Remplacez simplement le lien si votre domaine final est différent.

12.1 Version principale — à copier

Après plusieurs mois à construire, déployer et tester des applications, je suis heureux de mettre enfin mon portfolio en ligne.

On y trouve trois projets détaillés, avec le contexte, mon rôle et le résultat :

• **Signaleo** — signalement de pannes sur le terrain, suivi des interventions et supervision en temps réel.
• **E-Fandrosoana** — un ERP pour une quincaillerie : stocks, ventes, clients, facturation et trésorerie.
• **Pipeline CI/CD & observabilité** — une chaîne de livraison complète, de Docker à Prometheus/Grafana, avec déploiement blue/green.

Le site présente aussi mon parcours, mes certifications, des témoignages clients et un assistant IA qui répond aux questions sur mon profil.

Je suis à la recherche d'un **stage**, d'une **alternance** ou d'une **mission** en développement web/mobile, DevOps ou QA. Basé à Fianarantsoa, Madagascar, je suis ouvert au télétravail.

👉 **https://zo-mahefa-it.vercel.app/**

Vos retours sont les bienvenus. Et si vous connaissez une équipe qui cherche un profil polyvalent, produit + infrastructure + qualité, je serais ravi d'échanger.

12.2 Version courte — si vous préférez un post bref

Mon portfolio est en ligne. Développement web/mobile, DevOps et QA : trois projets détaillés (Signaleo, E-Fandrosoana, pipeline CI/CD), mes certifications, des témoignages clients et un assistant IA pour explorer mon profil.

Je cherche un stage, une alternance ou une mission — Fianarantsoa, Madagascar, remote friendly.

👉 https://zo-mahefa-it.vercel.app/

12.3 Hashtags

Ajoutez-les à la fin du post, sans en abuser (8 à 12 maximum) :

#PORTFOLIO #DÉVELOPPEURWEB #DEVOPS #QA #NEXTJS #SPRINGBOOT #MADAGASCAR #FIANARANTSOA #ALTERNANCE #STAGE #REMOTE

```
#Portfolio #DéveloppeurWeb #DevOps #QA #NextJS #SpringBoot #Madagascar #Fianarantsoa #Alternance
#Stage #Remote
```

12.4 Conseils de publication

- **Image** : joignez une capture du site ou l'image `og-image.png`. Un post avec visuel attire davantage l'attention.
- **Lien** : LinkedIn réduit parfois la portée des posts contenant un lien. Vous pouvez publier le texte avec l'image, puis mettre le lien dans le premier commentaire. Dans ce cas, écrivez « Lien en commentaire » dans le post.
- **Moment** : un mardi, mercredi ou jeudi matin (heure de Madagascar) est un créneau classique pour les posts professionnels.
- **Mention** : ne taguez pas des recruteurs ou des entreprises sans leur accord.
- **Vérité** : ne rajoutez pas de chiffres ou de clients que le portfolio ne peut pas prouver.
- **Réponses** : prévoyez de répondre aux commentaires le jour même ; c'est ce qui transforme une publication en conversation.
- **Réutilisation** : le même texte peut servir de description sur GitHub, dans un CV ou dans un e-mail de candidature, en l'adaptant.

> **APRÈS PUBLICATION**
> Ajoutez le lien du portfolio dans votre profil LinkedIn (section « Coordonnées » ou « Site web »), dans votre signature e-mail et sur votre CV. Un portfolio sert surtout quand il est facile à retrouver.

CHAPITRE 13

Annexes

13.1 Les dix erreurs à éviter

1. **Modifier seulement le français.** Le test de parité échouera, ou l'anglais affichera l'ancien texte.
2. **Chercher les projets en base.** Ils sont dans `content.ts`.
3. **Inventer une valeur de `visual`.** Le type n'accepte que trois valeurs ; ajoutez le rendu ou utilisez une galerie.
4. **Mettre une image externe sans `remotePatterns`.** `next/image` la refusera.
5. **Utiliser un nom de fichier avec des espaces ou des majuscules.** Vercel est sensible à la casse.
6. **Oublier de redéployer après un changement de variable.** Les variables ne sont pas toujours rechargées par un simple push.
7. **Mettre l'URL avec un slash final.** `siteUrl` le retire, mais autant écrire l'URL propre.
8. **Utiliser la base de production pour les previews.** Les tests polluent les vrais messages et les vraies visites.
9. **Committer `.env.local`.** Vérifiez `git status` avant chaque premier push.
0. **Croire qu'un test vert suffit.** Le formulaire, l'admin et l'assistant doivent être testés à la main après déploiement.

13.2 Commandes utiles

Commande	Usage
`npm install` / `npm ci`	Installer les dépendances.
`npm run dev`	Serveur de développement.
`npm run dev -- --port 3100`	Changer le port.
`npm test`	Tests unitaires.
`npm run lint`	ESLint.
`npx tsc --noEmit`	Vérification TypeScript.
`npm run build`	Build de production.
`npm run start`	Démarrer le build de production.
`npm run db:init`	Créer les tables et index.
`turso db list`	Lister les bases Turso.
`turso db show <db> --url`	Afficher l'URL libSQL.
`turso db tokens create <db>`	Créer un jeton.
`turso db shell <db>`	Ouvrir un shell SQL.
`turso db shell <db> ".dump"`	Exporter la base.
`git status`	Vérifier les fichiers avant commit.

13.3 Où modifier quoi — aide-mémoire

Besoin	Fichier / clé
Nom, e-mail, téléphone, réseaux, CV	`content.ts` → `site`

Besoin	Fichier / clé
Titre du hero, intro, boutons, stats	`content.ts` → `hero`
Rôles animés	`content.ts` → `roles`
Titres des sections	`content.ts` → `sections`
Projets vedettes	`content.ts` → `featuredProjects`
Projets d’archive	`content.ts` → `archive.projects`
Pôles d’expertise	`content.ts` → `practices`
Stack par niveaux	`content.ts` → `stack.levels`
Bandeau de technologies	`components/site/stack-marquee.tsx` → `const stack`
Bloc QA	`content.ts` → `qa`
Méthode	`content.ts` → `process`
Pipeline	`content.ts` → `pipeline`
Parcours, formation, certifications	`content.ts` → `about`
Témoignages	`content.ts` → `testimonials`
Contact et formulaire	`content.ts` → `contact`
Footer	`content.ts` → `footer`
Micro-libellés	`content.ts` → `ui`
Textes de l’assistant	`content.ts` → `ai`
Règles de l’assistant	`lib/assistant-prompt.ts`
SEO global	`app/layout.tsx`
Sitemap / robots	`app/sitemap.ts`, `app/robots.ts`
En-têtes de sécurité	`next.config.ts`
Schéma de base	`lib/db.ts`, `scripts/init-db.mjs`
Authentification admin	`lib/auth.ts`
Interface admin	`app/admin/page.tsx`

13.4 Maintenance recommandée

Fréquence	Action
À chaque contenu modifié	`npm test`, `npm run build`, vérification FR/EN, commit.
Chaque mois	Vérifier les messages reçus, supprimer les messages traités, consulter l’audience, tester le formulaire.
Chaque trimestre	Mettre à jour le CV, les projets, les certifications ; vérifier les liens GitHub/démo ; lancer `npm audit --omit=dev`.
Chaque semestre	Mettre à jour les dépendances avec prudence, revoir le SEO, rafraîchir l’image Open Graph, sauvegarder Turso.
En cas d’incident	Consulter les logs Vercel, vérifier Turso, tester `/api/messages` et `/api/visits`, faire un rollback si nécessaire.

13.5 Glossaire

Terme	Définition
App Router	Système de routage de Next.js basé sur le dossier `app/`.
SSR	Rendu côté serveur : le HTML est généré à la requête.

Terme	Définition
Hydratation	Processus par lequel React attache les interactions au HTML déjà rendu.
libSQL	Fork de SQLite utilisé par Turso, accessible en réseau.
Turso	Service de base de données libSQL hébergée.
HMAC	Signature cryptographique utilisée pour le cookie de session.
Rate limit	Limitation du nombre de requêtes sur une période.
Open Graph	Balises qui contrôlent l'aperçu d'un lien sur les réseaux sociaux.
JSON-LD	Données structurées pour les moteurs de recherche.
Preview Deployment	Déploiement Vercel d'une branche, avant fusion en production.

13.6 Fin du guide

Ce document décrit l'état du projet à sa date de rédaction. Si une procédure ou un fichier change, mettez à jour le guide en même temps que le code : c'est ce qui évite de se retrouver, six mois plus tard, devant une documentation qui ne correspond plus à la réalité.

Pour toute modification sensible — base de données, authentification, variables d'environnement — faites une sauvegarde et testez d'abord sur une base ou un déploiement de test.

Guide du portfolio — Zo Mahefa RANAIVO
Édition 2026 · v1.0 · Document généré pour la maintenance et le déploiement.
Aucun secret n'est inclus dans ce PDF. Les valeurs sensibles restent dans `.env.local` et dans les variables d'environnement Vercel.